
Topological Neural Operators

Lennart Bastian¹ Samuel Leventhal² Mustafa Hajji^{†,2} Tolga Birdal^{†,1}

¹Imperial College London, UK ²University of San Francisco, USA

Abstract

We introduce **Topological Neural Operators** (TNOs), a principled framework for operator learning on cell complexes that lifts neural operators (NOs) from functions on points to topological domains. TNOs represent data as features defined on cells of varying dimension and model their interactions through *Discrete Exterior Calculus*, enabling explicit cross-dimensional coupling via gradient-, curl-, and divergence-type operators. The key design principle is to decouple *where* information flows—governed by fixed topological operators—from *how* it is transformed (which is learned), yielding models that respect the geometric support of physical quantities and expose conservation and compatibility structure. We further propose **Hierarchical TNOs** (HTNOs), which incorporate learned coarse complexes to propagate long-range and topology-dependent information. Our framework subsumes existing NOs as a special case, providing a unified perspective on operator learning across discretizations. Across a range of PDE benchmarks, TNOs and HTNOs achieve improved accuracy while exhibiting stronger physical consistency in structured settings.

1 Introduction

Modeling and predicting complex physical systems is a central challenge in science and engineering. Many systems, from fluid dynamics to electromagnetism and elasticity, are governed by *partial differential equations* (PDEs), whose solutions under varying parameters, geometries, and boundary conditions underpin applications such as digital twins, design optimization, and real-time simulation [8]. Classical numerical methods (e.g., finite elements, spectral solvers) are accurate but computationally costly, often requiring fine discretizations and multiple solves/iterations.

Operator learning addresses this by learning the solution itself: a map between infinite-dimensional function spaces that captures how PDE solutions depend on inputs such as coefficients, forcing terms, and geometry [19, 47, 46, 60]. *Neural operators* (NOs) [48] have emerged as a powerful instantiation of this idea, achieving strong performance and generalization across discretizations and resolutions via variants such as Fourier Neural Operators (FNOs) [52, 59] and their geometric or transformer-based extensions [39, 1]. These methods enable orders-of-magnitude speedups over classical solvers while maintaining high fidelity, establishing a scalable paradigm for scientific machine learning.

Despite these successes, existing NOs are fundamentally *point-centric*: they represent all physical quantities as functions defined on nodes, with interactions modeled via learned kernels or graph-based message passing. This abstraction neglects a key structural aspect of continuum physics: *physical quantities have geometric type* [44]. Potentials live on vertices, circulations on edges, fluxes on faces, and densities in volumes; their interactions are governed by differential operators like gradient, curl, and divergence, that couple quantities across dimensions. In compatible numerical discretizations, these relationships are encoded directly through incidence structures, ensuring conservation laws and identities (e.g., $\text{div curl} = 0$) hold by construction¹. By collapsing all quantities to nodes, current

[†]Equal senior authorship.

¹The boundary-of-boundary is zero: a boundary shape (like a compact/closed surface) has no boundary of its own.

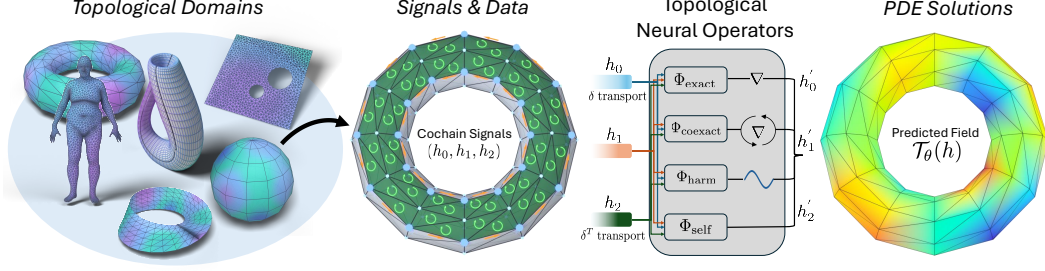


Figure 1: **Topological Neural Operators** operate on cell complexes (i) whose physical signals are cochains at multiple ranks (ii). A TNO layer (iii) couples ranks through fixed DEC operators (d^k , δ^k) and learns the rank-wise channel mixing in four blocks, signifying gradient, curl, harmonic, and self maps, producing a predicted PDE field on the complex (iv).

neural operators obscure this structure, limiting their ability to model multi-physics systems, enforce conservation and compatibility, and capture topological effects such as cycles and harmonic modes.

As a remedy, we introduce **Topological Neural Operators (TNOs)**, a principled framework for operator learning on *cell complexes*, the natural discrete domain for multi-dimensional physical systems (see Fig. 1). Our key design principle is to separate where information flows, governed by fixed topological operators, from how it is transformed, which is learned. Rather than representing data solely on points (or edges), TNOs operate on *cochains*, assigning features to cells of different dimensions (vertices, edges, faces, and higher-dimensional elements), and model their interactions using *Discrete Exterior Calculus* (DEC) [44]. This design explicitly encodes cross-dimensional structure: information flows between cells through incidence-based operators, while learnable components act on feature transformations. As a result, TNOs (i) respect the geometric support of physical quantities, (ii) incorporate conservation and compatibility structure into the architecture, and (iii) enable explicit modeling of multi-degree PDE systems.

This motivates a minimal extension of the NO paradigm: keep operator learning, but replace rank-0 point fields by cochain fields supported on cells of different dimensions. TNOs recover standard point-based NOs when restricted to 0-cochains, while allowing vertex-, edge-, face-, and volume-supported fields to interact through DEC operators. To capture long-range and topology-dependent effects, we further introduce **Hierarchical TNOs (HTNOs)**, which propagate information across learned coarse complexes while preserving the cochain structure. In brief, **our contributions are**:

- **A framework for operator learning on cell complexes**, extending neural operators to cochain-valued fields and enabling explicit cross-dimensional modeling via DEC.
- **A unifying perspective** encapsulating existing NOs as special cases of the proposed framework.
- **Hierarchical HTNOs** for efficient multi-scale propagation of global and topological information.
- **Broad ablation studies** validating the design choices, including PDEs with native multi-rank structure, showing that TNOs can separate and couple physical signals across cochain degrees.

We validate our approach across synthetic and real PDE benchmarks. (H)TNOs achieve improved accuracy and generalization across meshes, while exhibiting stronger physical consistency in structured settings. We will make our implementation publicly available upon publication.

2 Background and the Language of Discrete Topology

Physical quantities are dimensional: scalars live on vertices, circulations on edges, fluxes on faces, densities in volumes. The laws relating them are statements about boundaries, orientations, and dimensional couplings; this is the language of discrete exterior calculus used throughout. We provide the necessary background and refer the reader to [31, 30, 44, 32, 61, 33] for a more thorough treatment on DEC.

2.1 Topological Structures

Definition 2.1 (Regular Cell Complex). *A regular cell complex is a finite set K together with a rank function $\text{rk} : K \rightarrow \mathbb{Z}_{\geq 0}$ and a face relation $\sigma \prec \tau$, defined only when $\text{rk}(\tau) - \text{rk}(\sigma) = 1$, such that for any $\tau, \rho \in K$ with $\text{rk}(\tau) - \text{rk}(\rho) = 2$, exactly two elements $\sigma \in K$ satisfy both $\rho \prec \sigma$ and $\sigma \prec \tau$.*

Elements of K are called *cells*; those of rank k form the set K_k , with $n_k = |K_k|$ and $N = \max_\tau \text{rk}(\tau)$. Rank-0 cells are vertices, rank-1 edges, rank-2 faces, and rank-3 volumes. This structure is the correct domain language for physics: quantities such as pressure, circulation, magnetic flux, and charge density live on cells of rank 0, 1, 2, 3 respectively (see Tab. 4). Every k -cell is a first-class computational entity able to exchange information with any incident cell. As we show in Sec. 3, the governing equations of Maxwell, Navier–Stokes, and elasticity are precisely statements about how quantities at different ranks interact, a structure graphs cannot represent and cell complexes encode natively. In the following, we abbreviate regular cell complexes as RCC.

The face relation $\sigma \prec \tau$ records immediate incidence and determines differential operators. For every k -cell τ and $(k-1)$ -cell σ one assigns an orientation sign $[\tau : \sigma] \in \{-1, 0, +1\}$, with $[\tau : \sigma] \neq 0$ iff $\sigma \prec \tau$. Assembling these into the *signed incidence matrix* $B_k \in \mathbb{R}^{n_{k-1} \times n_k}$ with $(B_k)_{\sigma\tau} = [\tau : \sigma]$, the *diamond condition* [10, 7, 75] forces

$$B_k B_{k+1} = 0 \quad \forall k, \quad (1)$$

the discrete statement that **the boundary of a boundary is empty**, which underlies $\text{curl}(\text{grad}) = 0$ and $\text{div}(\text{curl}) = 0$, and is the structural foundation of everything that follows.

A k -cochain $u^k : K_k \rightarrow \mathbb{R}^{d_k}$ assigns a feature vector to each k -cell; the cochain space is $C^k(K; \mathbb{R}^{d_k}) \cong \mathbb{R}^{n_k \times d_k}$. The degree k is geometrically meaningful: it records the type of object on which the quantity lives, and assigning a field to the wrong degree breaks the structural laws governing it (see Sec. A.2). The full multirank signal space is $C^\bullet = \bigoplus_{k=0}^N C^k$.

In practice, we also use more flexible Combinatorial Complexes [36, 35], which generalize graphs, RCCs, and hypergraphs by allowing cells at different ranks to be specified independently rather than constrained to arise as boundaries of higher-rank cells. We introduce these in Sec. 4.

2.2 Discrete Exterior Calculus

From the boundary matrices, one derives three families of operators that serve as the computational primitives of the Discrete Exterior Calculus (DEC) framework [31, 30, 33].

Discrete exterior derivative. The *discrete exterior derivative* $d^k = B_{k+1}^\top : C^k \rightarrow C^{k+1}$ lifts a k -cochain to a $(k+1)$ -cochain by accumulating signed values over incidence relations. Concretely, d^0 is the discrete gradient (oriented differences across edges), d^1 is the discrete curl (oriented sums around faces), and d^2 is the discrete divergence-of-dual (oriented sums over bounding faces of a volume). The identity $d^{k+1} \circ d^k = 0$ follows directly from Eq. (1) and is the discrete counterpart of $\text{curl}(\text{grad}) = 0$ and $\text{div}(\text{curl}) = 0$. See Section A.2 for details.

Hodge operators and codifferential. Equipping each C^k with a positive-definite *Hodge star*² $M_k \in \mathbb{R}^{n_k \times n_k}$ encoding geometric data such as edge lengths, face areas, and cell volumes, gives the formal M_k -adjoint of d^{k-1} , the *codifferential*, mapping downward in degree:

$$\delta^k = M_{k-1}^{-1} B_k M_k : C^k \rightarrow C^{k-1}. \quad (2)$$

While d^k is purely topological (determined by B_{k+1} alone), δ^k depends on geometry encoded in M_k .

Hodge Laplacian. Together, d^k and δ^k yield the k -Hodge Laplacian

$$\Delta_k = \underbrace{\delta^{k+1} \circ d^k}_{\Delta_k^\uparrow} + \underbrace{d^{k-1} \circ \delta^k}_{\Delta_k^\downarrow} : C^k \rightarrow C^k, \quad (3)$$

which is self-adjoint and positive semidefinite with respect to M_k . The decomposition into $\Delta_k^\uparrow$ and $\Delta_k^\downarrow$ is physically meaningful: $\Delta_k^\uparrow$ couples u^k upward to $(k+1)$ -cells via d^k (curl-like coupling), while $\Delta_k^\downarrow$ couples u^k downward to $(k-1)$ -cells via δ^k (divergence-like coupling). At $k=0$, $\Delta_0 = \delta^1 d^0$ recovers the weighted graph Laplacian; at $k=1$, Δ_1 simultaneously involves edges, vertices, and faces, an already operator with no graph-level analog [30].

Hodge Decomposition (HD). The three operator families organize every cochain space into an orthogonal decomposition with respect to the M_k -inner product:

$$C^k(K; \mathbb{R}) = \underbrace{\text{im}(d^{k-1})}_{\text{exact}} \oplus \underbrace{\text{ker}(\Delta_k)}_{\text{harmonic}} \oplus \underbrace{\text{im}(\delta^{k+1})}_{\text{coexact}}, \quad (4)$$

²See Section A.2 for the precise definition.

Exact cochains are *potential-driven*; coexact cochains are *divergence-free*; harmonic cochains are *topologically constrained*, with

$$\dim \ker(\Delta_k) = \beta_k \quad (5)$$

where β_k is the k -th Betti number, counting independent k -dimensional holes [33].

Hodge Decomposition is central to our TNO for two reasons: it characterizes what physical solutions look like (incompressible flows are coexact; conservative fields are exact; topological modes are harmonic), and it provides the DEC-compatible channels through which conservation and compatibility constraints can be preserved, as we show in Sec. 4.1.

The complex K thus encodes in a single structure everything a learning framework needs: the domain $\{K_k\}_{k=0}^N$ stratified by dimension, the signal spaces $\{C^k(K; \mathbb{R}^{d_k})\}$ where physical fields live, the differential operators $d^k = B_{k+1}^\top$, $\delta^k = M_{k-1}^{-1} B_k M_k$, and $\Delta_k = \delta^{k+1} d^k + d^{k-1} \delta^k$ that move information between dimensions, and the $B_k B_{k+1} = 0$ that encodes the algebraic backbone of discrete physical laws.

3 Discrete PDEs on Cell Complexes

The operators d^k , δ^k , Δ_k appear directly in the governing equations of physical systems, and most of these systems couple fields across more than one degree.

As in Sec. 2, let K be a finite cell complex of dimension N equipped with Hodge stars $\{M_k\}_{k=0}^N$. The de Rham complex gives a common discrete language for physical PDEs on bounded domains: a field is stored on the cells where it naturally lives, while d^k , δ^k , and Δ_k describe how that field changes across neighboring dimensions [33, 5]. Thus scalar potentials live on vertices, circulations on edges, fluxes on faces, and densities on volumes.

A discrete PDE on $(K, \{M_k\})$ at degree k is an equation

$$\mathcal{F}(u^k, d^k u^k, \delta^k u^k, \Delta_k u^k, a, f) = 0 \quad \text{in } C^k(K; \mathbb{R}^d), \quad (\text{PDE})$$

where $u^k \in C^k(K; \mathbb{R}^d)$ is the unknown field and $f \in C^k(K; \mathbb{R}^d)$ is the source. The coefficient field $a \in C^\bullet(K; \mathbb{R}^p)$ contains the material or constitutive data that define the particular PDE instance, such as diffusion coefficients, conductivity, permeability, permittivity, or density. It may live on one degree or across several degrees, depending on the physics. The map $\mathcal{F}$ is the PDE residual; solving the PDE means finding u^k so that this residual vanishes.

Boundary conditions are imposed on a boundary subcomplex $\partial K \subseteq K$. Essential conditions prescribe the boundary trace $u^k|_{\partial K}$, while natural conditions prescribe flux-type data and enter through the Hodge-adjoint codifferential δ^k , hence through the boundary-modified incidence and Hodge-star operators.

Many physical systems involve more than one degree at once: Maxwell equations couple a 1-form with a 2-form, Darcy couples a 0-form with a 1-form, Navier–Stokes in vorticity form couples 1- and 2-forms. A *multi-degree discrete PDE* is a system

$$\mathcal{F}_k(\{u^j\}_{j=0}^N, \{d^j u^j\}_j, \{\delta^j u^j\}_j, a, f) = 0 \quad \text{in } C^k(K; \mathbb{R}^{d_k}), \quad k = 0, \dots, N, \quad (\text{mPDE})$$

with unknown $u \in C^\bullet(K; \mathbb{R}^{d_\bullet})$. The equation at degree k can depend on $d^{k-1} u^{k-1}$ (information from below, via $\Delta_k^\downarrow$ -type coupling), on $\delta^{k+1} u^{k+1}$ (from above, via $\Delta_k^\uparrow$ -type coupling), and on $\Delta_k u^k$. The coupling pattern is fixed by the cellular structure of K ; the weights, by $\{M_k\}$. A learnable map that processes degrees independently, without d and δ connecting them, cannot represent this.

For time-dependent problems, the unknown is $u : [0, T] \rightarrow C^\bullet(K; \mathbb{R}^{d_\bullet})$ satisfying

$$\partial_t u^k(t) = \mathcal{A}_k(\{u^j(t)\}_{j=0}^N, \{d^j u^j(t)\}_j, \{\delta^j u^j(t)\}_j, a, f(t)), \quad u(0) = u_0, \quad (\text{Evol})$$

encompassing parabolic / hyperbolic problems, and fully coupled multi-physics systems. Discretizing in time yields one-step maps $u(t) \mapsto u(t + \Delta t)$, the natural targets for time-stepping operators.

Examples. In the Sec. A.3.1, we illustrate (Evol) with two standard systems whose fields live on different cochain degrees and couple through d and δ : discrete Maxwell- and wave-type couplings. In both cases, the dynamics are incidence-driven: fields on different cochain degrees are coupled by d and δ , and the associated constraints are preserved by the algebraic identities of the complex.

4 Topological Neural Operators (TNOs)

Each PDE admits a *solution operator* sending input data (initial conditions, sources, parameters) to the output field: for the wave system, a map $C^0 \times C^1 \rightarrow C^0 \times C^1$ advancing (u, p) in time; for Maxwell, a map $C^1 \times C^2 \rightarrow C^1 \times C^2$ advancing (E, B) . Approximating such an operator $\mathcal{G}$ from data is the subject of operator learning [48]. Here $\mathcal{G}$ acts on tuples of cochains organized by degree, with its action mediated by d, δ, Δ on K . The following defines our class of learnable maps:

Definition 4.1 (Topological Neural Operators). *Let K be a finite cell complex of dimension N with boundary matrices $\{B_k\}$ and Hodge stars $\{M_k\}_{k=0}^N$, inducing discrete operators $\{d^k\}, \{\delta^k\}, \{\Delta_k\}$. For $i = 1, \dots, m$ and $j = 1, \dots, n$, let $k_i, \ell_j \in \{0, \dots, N\}$ and $d_i, r_j \geq 1$, and define*

$$\mathcal{X}(K) = \prod_{i=1}^m C^{k_i}(K; \mathbb{R}^{d_i}), \quad \mathcal{Y}(K) = \prod_{j=1}^n C^{\ell_j}(K; \mathbb{R}^{r_j}). \quad (6)$$

A Topological Neural Operator (TNO) is a parameterized family of neural networks $\{\mathcal{T}_\theta^K : \mathcal{X}(K) \rightarrow \mathcal{Y}(K)\}_{\theta \in \Theta}$ approximating a target operator $\mathcal{G} : \mathcal{X}(K) \rightarrow \mathcal{Y}(K)$ and satisfying:

- (P1) **Cellular intrinsicness.** $\mathcal{T}_\theta^K$ is built from the cellular operators of K , including boundary / coboundary maps, Hodge stars, codifferentials, Hodge Laplacians, trace / restriction maps, pointwise nonlinearities, and learned channel-mixing maps shared across cells of the same degree. Thus it depends on incidence and metric data, not on a fixed grid or cell ordering.
- (P2) **Multi-degree coupling.** Information may move between cochain degrees through boundary, coboundary, codifferential, Laplacian, and learned composed operators. Thus an output at one degree may depend on inputs at any other degree.
- (P3) **Discretization transferability.** The weights θ are shared across discretizations. If K' refines K , then lifting to K' (prolonging cochains via $P_{K \rightarrow K'}$), applying the operator, and restricting back (via $R_{K' \rightarrow K}$) should agree with applying the operator on K :

$$R_{K' \rightarrow K} \mathcal{T}_\theta^{K'} P_{K \rightarrow K'} x \approx \mathcal{T}_\theta^K x. \quad (7)$$

Remarks. (P1) makes the TNO intrinsic to the cellular discretization: incidence enters through the boundary maps, while geometric information enters through the Hodge stars, not through an extrinsic grid or an ordering of cells. (P2) is the key higher-order feature: signals may live on vertices, edges, faces, and higher cells, and the operator can couple these degrees. Such cross-degree coupling is not merely an architectural extension, but is imposed by the physics (e.g. Maxwell/wave PDEs in Sec. A.3.1). (P3) is the analogue of discretization consistency in NOs [48]: shared weights act across refinements.

The choice of degrees $\{k_i\}$ and $\{\ell_j\}$ in Dfn. 4.1 determines the operator type: degree-preserving ($C^k \rightarrow C^k$) for single-field PDEs; gradient-type ($C^0 \rightarrow C^1$) for potential-to-flux maps like Darcy or Fourier; divergence-type ($C^1 \rightarrow C^0$) for flux-to-source maps; and multi-degree ($C^0 \times C^1 \times C^2 \rightarrow C^0 \times C^1 \times C^2$) for coupled systems like Maxwell. We next detail how $\mathcal{T}_\theta$ is computed (*learned*).

4.1 Realization as a Topological Neural Network

Dfn. 4.1 specifies what a TNO must be (P1–P3); it does not say how to compute one. We now present one possible architecture that realizes these axioms as a topological neural network (TNN) [36].

Architecture. The TNO of Dfn. 4.1 is posed on an RCC K . For computation, it is convenient to lift K to a combinatorial complex (CC) rather than an RCC. A TNN realizing $\mathcal{T}_\theta^K : \mathcal{X}(K) \rightarrow \mathcal{Y}(K)$ has the form

$$\mathcal{T}_\theta^K = \text{Dec}_\theta \circ \mathcal{L}_\theta^{(L-1)} \circ \dots \circ \mathcal{L}_\theta^{(0)} \circ \text{Enc}_\theta. \quad (8)$$

The encoder lifts the input cochains on K to hidden cochains $h_0^k \in C^k(\tilde{K}; \mathbb{R}^{d_h})$ at every rank $k = 0, \dots, \dim \tilde{K}$. The incidence structure of $\tilde{K}$ supplies the discrete operators used in the network. The topological layers $\mathcal{L}_\theta^{(\ell)}$ update these cochains on $\tilde{K}$ while preserving their cochain-valued structure. The decoder reads the target degrees from the final hidden cochains and restricts to K .

The encoder and decoder bridge between K and $\tilde{K}$. Material parameters and other coefficient fields enter as input cochains, which keep the architecture intrinsic to $(\tilde{K}, \{M_k\})$ in the sense of (P1). Next, we explain how the individual topological layers $\mathcal{L}_\theta^{(\ell)}$ are realized.

4.2 Topological layers

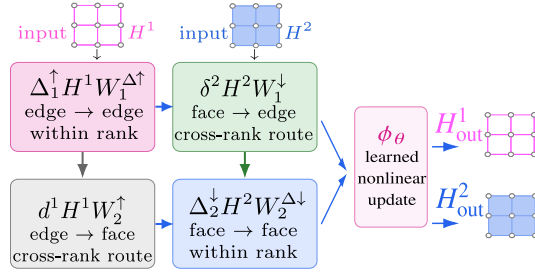
Each topological layer updates the hidden cochains on the lifted complex $\tilde{K}$ while preserving their geometric type. Let $\mathbf{H} = (H^0, \dots, H^N)$ with $H^k \in \mathbb{R}^{n_k \times d_h}$. The layer applies a learned nonlinear map ϕ_θ to a fixed block-structured operator that encodes the discrete exterior calculus of $\tilde{K}$. As shown on the right, $\mathbf{H}_{\text{out}}$ is defined through incidence matrices and Hodge stars of $\tilde{K}$. Its tridiagonal sparsity is exact: equations at degree k couple only ranks $k \pm 1$. The trainable objects are the channel-mixing matrices W and transformations ϕ_θ . In the rigid DEC version, the transport maps themselves are fixed; in the copresheaf variant [37] below, the incidence support remains fixed, but the fiber maps carried along incidences may be learned. See Appendix C.3.

$$\mathbf{H}_{\text{out}} = \phi_\theta \left(\begin{bmatrix} \Delta_0 & \delta^1 & 0 & \cdots \\ d^0 & \Delta_1 & \delta^2 & \cdots \\ 0 & d^1 & \Delta_2 & \ddots \\ \vdots & \vdots & \ddots & \ddots \end{bmatrix} \mathbf{H} W \right)$$

When only a single rank k is active, the block form reduces to (out-of-range terms omitted):

$$H_{\text{out}}^k = \phi_\theta \left(H^k, d^{k-1} H^{k-1} W_k^\downarrow, \delta^{k+1} H^{k+1} W_k^\uparrow, \Delta_k^\uparrow H^k W_k^{\Delta,\uparrow}, \Delta_k^\downarrow H^k W_k^{\Delta,\downarrow} \right). \quad (9)$$

Fig. 2 illustrates the template nature of TNOs: Maxwell fixes the active degrees and DEC routes, while the network learns how to mix the routed edge and face cochains (see App. B). The construction admits a continuous *rigidity spectrum*. At the rigid extreme, the transport operators are exactly the DEC operators of $\tilde{K}$. At the flexible extreme, each incidence $y \prec x$ carries a learnable fiber map $\rho_{y \rightarrow x}$, yielding twisted operators $d_\rho, \delta_\rho, \Delta_{\rho,k}$ (copresheaf realization).



Operator-learning interpretation The distinction from a generic CCNN layer [36] is that the support of spatial transport is prescribed by the DEC operators. Cross-rank coupling is carried by d and δ , while same-rank propagation is carried by $\Delta^\uparrow$ and $\Delta^\downarrow$. Thus, the architecture does not learn where information can flow; it learns how transported features are mixed through $W^\bullet$ and ϕ_θ . This is the architectural mechanism behind discretization transfer. Under compatible refinements and stable Hodge discretizations, the DEC operators d, δ, Δ approximate their continuum counterparts in the FEEC sense [3]. Since the same weights θ are shared across discretizations with support maps recomputed from each cell complex; the resulting family $\{\mathcal{T}_\theta^K\}_K$ is biased toward a common continuum rather than a discretization-specific message passing rule.

Figure 2: Visual TNO layer induced by Maxwell-type routing. Cross-rank transport is carried by d^1 and δ^2 , while same-rank propagation is carried by the Hodge-Laplacian channels $\Delta^\uparrow$ and $\Delta^\downarrow$. The learned weights $W^\bullet$ and nonlinear update ϕ_θ determine how strongly the routed features are mixed.

Linear realization of TNO. The simplest special case takes ϕ_θ to be linear and uses the Hodge decomposition channels directly. The rank- k linear TNO layer is

$$\mathcal{T}_{\theta,k}(h) = W_k^\uparrow \delta^{k+1} h^{k+1} + W_k^\downarrow d^{k-1} h^{k-1} + W_k^{\text{harm}} P_k^{\text{harm}} h^k + W_k^{\text{self}} h^k. \quad (10)$$

The four terms correspond to coexact, exact, harmonic, and local channels: $\text{im}(\delta^{k+1})$, $\text{im}(d^{k-1})$, $\ker(\Delta_k)$, and a residual self-channel, respectively. Thus, the layer separates the three topological components of a rank- k signal while keeping transport fixed by the DEC operators. The following propositions show how FNO and GNO arise as rank-0 specializations of the TNO framework.

Proposition 4.2 (FNO as a rank-0 spectral TNO [52]). *Let K_N be a uniform periodic cubical complex with $N_1 \times \dots \times N_d$ vertices, and let $u \in C^0(K_N; \mathbb{R}^{c_{\text{in}}})$. Let $\mathcal{F}_N$ denote the discrete Fourier transform on 0-cochains. A linear FNO layer*

$$u \mapsto W u + \mathcal{F}_N^{-1} R_\theta \mathcal{F}_N u \quad (11)$$

is realized by a rank-0 spectral TNO layer whose same-rank spectral channel is diagonal in the Fourier basis of K_N . Proof is provided in Sec. C.1.

Proposition 4.3 (GNO as a rank-0 graph-quadrature TNO [51]). *On a directed graph complex whose vertices are sample points with rank-1 cells the quadrature neighborhoods, the GNO update :*

$$u_i \mapsto W u_i + \sum_{j \in \mathcal{N}(i)} \kappa_\theta(x_i, x_j) u_j \omega_j, \quad (12)$$

Table 1: Established steady-state benchmarks [53, 40] as used in [64]. We depict test relative L^1 (%) error. **Bold**: best; underline: second. [†] our runs with official implementations (see Sec. F.2).

Dataset	Ours		Baselines							
	HTNO	TNO	RIGNO-18 [64]	RIGNO-12 [64]	GAOT [†] [86]	MGN [69]	Geo-FNO [53]	FNO DSE [57]	GINO [54]	UPT [1]
Poisson-Gauss	<u>1.30</u>	1.03	2.26	2.52	1.39	30.9	8.16	2.27	7.57	48.4
Airfoil	1.21	1.11	1.00	<u>1.09</u>	3.12	10.1	4.48	1.99	2.00	45.7
Elasticity	1.70	<u>2.73</u>	4.31	4.63	4.07	11.9	5.53	4.81	4.38	12.6

is recovered as a rank-0 TNO with a learned incidence-supported kernel message $h^1(e_{ij}) = \kappa_\theta(x_i, x_j)u_j\omega_j$ on each directed rank-1 cell $e_{ij} : v_j \rightarrow v_i$, followed by target-incidence aggregation at each vertex. Proof is provided in Sec. C.2.

Hierarchical Realization of TNO (HTNO). The TNO construction extends naturally to a hierarchy of cell complexes. Let $K_0 \leftarrow K_1 \leftarrow \dots \leftarrow K_L$, where $K_{\ell+1}$ is a coarsening of K_ℓ and $K_0 = K$. Each level carries cochain spaces $C^k(K_\ell)$ and discrete exterior derivatives $d_\ell^k : C^k(K_\ell) \rightarrow C^{k+1}(K_\ell)$. Levels are connected by degree-preserving transfer maps

$$R_\ell^k : C^k(K_\ell) \rightarrow C^k(K_{\ell+1}), \quad \Pi_\ell^k : C^k(K_{\ell+1}) \rightarrow C^k(K_\ell). \quad (13)$$

Ideally, these transfers commute with the coboundary:

$$d_{\ell+1}^k R_\ell^k = R_\ell^{k+1} d_\ell^k, \quad d_\ell^k \Pi_\ell^k = \Pi_\ell^{k+1} d_{\ell+1}^k. \quad (14)$$

This is the analogue of the commuting-diagram condition in FEEC multigrid [5, 3]. When (14) holds, exact components are preserved across levels, so transfer respects the de Rham structure rather than mixing cochain types arbitrarily. Coexact and harmonic components are controlled through the corresponding Hodge-adjoint and Hodge-Laplacian structure induced by the level-wise Hodge stars. For learned coarsenings, such as soft-clustering or soft Voronoi partitions, exact commutation is generally not guaranteed; in that case, (14) is an architectural bias or regularization target rather than a strict algebraic guarantee.

An HTNO arranges TNO blocks as a learned V -cycle for de Rham complexes [41, 82]: TNO layers on K_ℓ are Hodge-compatible smoothers [42], with an additive coarse-grid correction transferred via R_ℓ^k, Π_ℓ^k that come from a partition of K_ℓ — either fixed (k -means) or learned end-to-end as a soft Voronoi with trainable centroids. The operator class is unchanged: it is still a TNO at every level — (P1)–(P3) hold level-wise — with fine blocks capturing local cross-degree physics and coarse blocks propagating long-range information. Sec. D gives the V -cycle equation and implementation.

5 Experiments

We evaluate TNO and HTNO on a broad span of steady-state PDEs over irregular 2D and 3D meshes: Poisson’s equation with Gaussian and multiscale-sinusoidal sources, hyper-elastic deformations of variable domains, compressible flow past 2D airfoils spanning subsonic, transonic, and super-sonic regimes, and large-scale 3D wing-surface aerodynamics under compressible RANS in the transonic regime. These come from three established public suites [64, 86, 66], with mesh sizes ranging from $\sim 1K$ to 65K nodes. We complement these with two controlled studies of our own: an anisotropic-Darcy experiment with per-face random tensor orientation that isolates how TNO natively ingests higher rank signals (Sec. 5.3), and a component ablation on synthetic topologies (Darcy with reaction; conservative advection–diffusion) that disentangles the harmonic-basis input and sheaf transport (Sec. 5.4). Datasets descriptions and full training protocol are deferred to Sec. F. Implementations are in JAX and run on GH200 chips (96GB VRAM).

5.1 Steady-State Benchmarks on Irregular Geometry

Poisson-Gauss, Airfoil, Elasticity (Tab. 1). We evaluate on three established steady-state benchmarks assembled by [64]: Poisson-Gauss (PG, fixed grid; from [40]), Airfoil Flow (AF, variable geometry), and Elasticity (variable; both from [53]). We compare against the seven baselines reported in [64]: RIGNO-18, RIGNO-12, MeshGraphNet (MGN) [69], Geo-FNO [53], FNO DSE [57], GINO [54], and UPT [1]. (see Tab. 1). Both TNO and HTNO outperform all seven on PG and Elasticity; TNO is competitive with RIGNO-18 on AF, which fixes the far-field at $M_\infty=0.8$, $\alpha=0^\circ$ and varies only the airfoil geometry [53], so methods saturate near the discretization noise floor; the GAOT

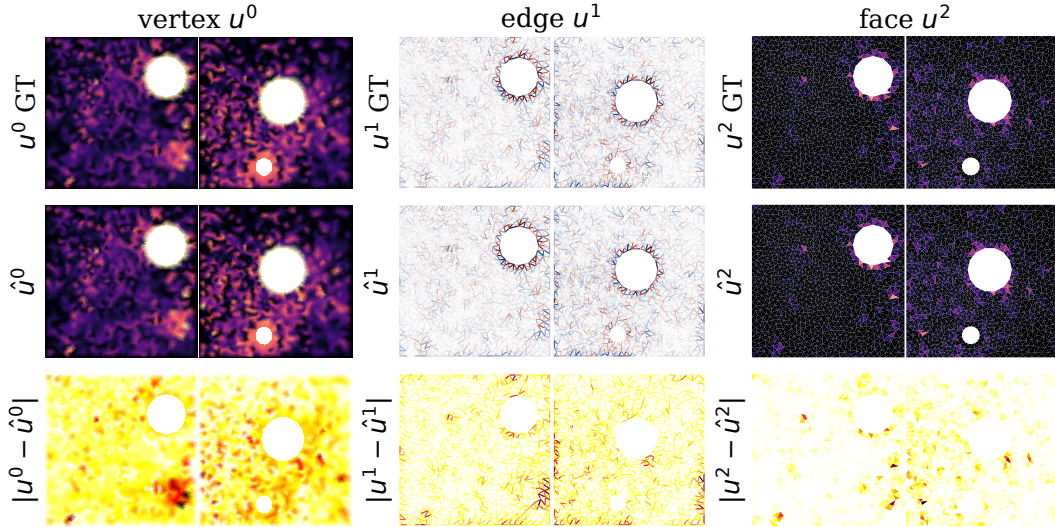


Figure 3: **Qualitative results for Anisotropic Darcy** with per-face random tensor orientations. Coupled signals require disentangling physical quantities at multiple topological ranks simultaneously.

Table 2: Additional steady-state benchmarks: test relative L^1 (%). † Runs with official implementation; **bold**: best, underline: second. EmmiWing is given per-variable (p_s , $\tau_{x,y,z}$) and aggregate (Agg.).

(a) Compressible airfoils and PCS [86]					(b) EmmiWing [66].					
Dataset	Ours		Baselines		Model	p_s	τ_x	τ_y	τ_z	Agg.
	HTNO	TNO	RIGNO †	GAOT †						
NACA0012	4.53	3.77	5.09	12.13	PointNet †	0.29	4.24	5.75	5.90	2.62
NACA2412	<u>4.73</u>	3.92	5.73	12.25	Transformer †	0.30	4.21	5.57	6.13	2.60
RAE2822	<u>4.55</u>	3.92	4.81	12.98	RIGNO-18 †	0.29	4.46	6.02	6.30	2.78
PCS	0.52	3.09	7.11	<u>0.72</u>	Transolver †	<u>0.28</u>	4.14	<u>5.52</u>	<u>5.57</u>	<u>2.55</u>
					HTNO	0.24	3.94	5.32	5.37	2.41

airfoils (Tab. 2a) genuinely sweep $M \in [0.5, 1.4]$ and $\alpha \in [0.5^\circ, 5.0^\circ]$ per sample [86], crossing the shock-formation boundary (see Sec. G.2).

Compressible airfoils and Poisson-with-sines (Tab. 2a). We additionally evaluate on the four single-file steady-state datasets [86]: three compressible-airfoil sets (NACA0012/2412, RAE2822) on 8K-node meshes, spanning subsonic to supersonic regimes, and Poisson-with-sines on a fixed 16K-node mesh (PCS). On all three airfoils, both TNO and HTNO clearly outperform RIGNO and GAOT. This can be attributed to the elliptic characteristics that dominate steady compressible flow at subsonic and transonic conditions (smooth pressure/velocity outside shocks), which align naturally with the Hodge / harmonic-basis inductive biases that characterize TNOs. PCS is the converse setting: a fixed mesh with a smooth, multiscale-sinusoidal Poisson source, for which GAOT’s structured-latent encoder is well matched, beating RIGNO by an order of magnitude. Nevertheless, the HTNO, which combines local and global interactions, yields strong performance (**0.52** vs. 0.72).

5.2 Large-Scale Surface PDEs: EmmiWing

EmmiWing [66] comprises 3D wing surfaces (raw meshes 244K–426K nodes; variable per sample) with four output channels (surface pressure p_s plus three wall-shear-stress components $\tau_{x,y,z}$; official 25,674/999/2,992 split). Following the AB-UPT training regime [2], we train on 65K shared FPS query points per sample with 16,384-node random subsampling per step, and evaluate uniformly on the 65K FPS subset rather than the full raw mesh; see Sec. F.3 for the full protocol. HTNO leads aggregate L^1 at 2.41%, beating Transolver [87] (2.55%), our matched-budget self-attention Transformer (2.60%) and PointNet (2.62%), and RIGNO-18 (2.78%); HTNO additionally converges in $\sim 6\times$ fewer epochs than RIGNO-18. TNO is subsumed by the hierarchical HTNO, which models long-range interactions without per-sample Hodge decompositions on large resampled geometries.

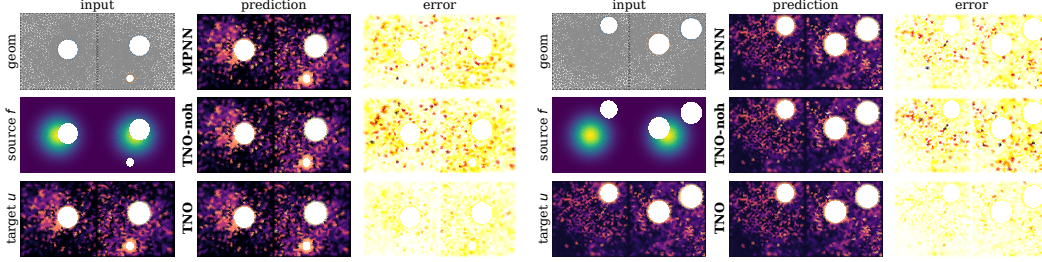


Figure 4: Qualitative ablation across three models: Vanilla MPNN vs. TNO (no harmonics) vs. TNO with harmonics / copresheaves on Anisotropic Darcy (left) and Advection Diffusion (right), for which we depict model inputs and targets (col. 1), model predictions (col. 2), and model errors (col. 3).

Table 3: Rank-input and component studies: test relative error (%); lower is better, **bold**: best, underline: second; *param-matched MPNN ($w=272$, $\sim 5.35\text{M} \approx \text{TNO } 5.0\text{M}$).

(a) Anisotropic Darcy with per-face random tensor orientation (Sec. 5.3); L^1 / L^2 . (b) TNO component ablation on synthetic topologies (Sec. 5.4); L^1 per PDE.

Variant	L^1	L^2
MPNN ($w=192$), vertex	9.97	7.03
MPNN ($w=192$), projected	9.10	6.52
MPNN ($w=272$)*, projected	7.59	5.28
TNO, vertex	<u>5.39</u>	3.85
TNO, projected	5.42	<u>3.66</u>
TNO, native rank-2	4.88	3.32

Variant	Darcy	Adv.-Diff.
MPNN ($w=192$)	8.87	9.78
MPNN ($w=272$)*	7.29	7.66
TNO	9.44	10.68
TNO + harm. basis	<u>5.16</u>	<u>5.77</u>
TNO + copresheaf	11.64	13.05
TNO + copresheaf + harm.	4.87	5.20

5.3 Anisotropic Darcy with Per-Face Random Tensor Orientation

Setup. TNOs naturally accommodate higher-rank signals. To validate this topology-aware inductive bias, we simulate the following anisotropic Darcy PDE on 100 triangulated planar square domains $[-1, 1]^2$ with one or two randomly placed circular holes:

$$-\nabla \cdot (\kappa(x) \nabla u) = f, \quad u|_{\partial\Omega_{\text{outer}}} = 0, \quad u|_{\partial\Omega_{\text{hole},k}} = g_k \sim \mathcal{U}[0.5, 1.5], \quad (15)$$

The diffusivity $\kappa(x)$ is a piecewise-constant face-valued symmetric tensor with eigenvalues $(\kappa_{\parallel}, \kappa_{\perp}) = (4, 1)$ and a per-face principal axis $\phi_t \sim \mathcal{U}[0, \pi)$ drawn iid per triangle and fixed across all samples on a given mesh, so that the rank-2 orientation field $\cos(2\phi_t)$ has unit-order variance on every face, but *vanishing population mean*. **Projection onto vertices is therefore lossy by construction:** averaging $\cos(2\phi_t)$ over incident faces concentrates to ≈ 0 at every vertex regardless of the underlying κ . This isolates the effect of *how* the orientation is utilised. An architecture that ingests it at its native rank 2 retains operator information that architectures isolated to vertices discard. We compare TNO (rank-2, harmonic basis, sheaf transport) against an MPNN baseline at matched and parameter-matched width (w), each consuming the orientation field in three ways: not at all (vertex inputs only), as a vertex-projected channel, or natively at rank 2. PDE simulation, training recipe, mesh construction, and face-supervision cells are in Sec. F.4.

Results. Two distinct effects emerge (see Tab. 3a). **(i) Architecture, $\sim 4\text{--}5$ pp, independent of how orientation is consumed.** TNO outperforms MPNN by 4.58 pp on vertex inputs and by 3.68 pp on projected inputs; even at parameter parity. The harmonic-basis channel resolves the spectral signature of the anisotropic operator, which vanilla message-passing cannot. **(ii) Native rank-2 ingestion, an additional ≈ 0.5 pp.** On the domain where vertex projection is lossy by design, TNO with native rank-2 orientation beats TNO with projected orientation by 0.54 pp ($\sim 10\%$ relative), while the projected and vertex-only variants are within 0.03 pp of each other, confirming that the projected channel carries no operator information, as demanded by the iid- ϕ construction. The two effects are additive: the architecture earns ~ 5 pp; native rank-2 ingestion yields another ~ 0.5 pp improvement.

5.4 Ablations

Setup. We isolate the contribution of TNO’s harmonic-basis input and sheaf transport on two scalar PDEs whose flux carries non-trivial curl on the same mesh family as Sec. 5.3: Darcy with reaction $(-\nabla \cdot (\kappa \nabla u) + \sigma u = f)$ and conservative advection–diffusion. Per-sample variability is restricted to

the per-hole Dirichlet BC values, so the topology is the only axis that varies across samples on a fixed mesh. All variants share a common backbone (hidden dim 192; run 300 epochs); MPNN baselines at matched width and parameter count bracket the comparison.

Results. Disabling the harmonic basis yields a loss of 6.8 pp on Darcy and 7.9 pp on Adv.-Diff (see Tab. 3b). Sheaf transport and the harmonic basis are synergistic, not additive: with harmonic on, sheaf helps; with harmonic off, sheaf actively hurts (Darcy $9.44 \rightarrow 11.64$). The complete TNO wins on both PDEs, beating the param-matched MPNN by $\sim 33\%$ (see Fig. 4). This implies harmonic and sheaf components are an effective inductive bias, particularly for PDEs with non-trivial curl.

6 Conclusion

We introduced **TNOs**, a topological framework for operator learning on cell complexes. Fields are represented as cochains on vertices, edges, faces, and volumes, and their interactions are routed by DEC operators. Thus topology determines where information flows, while learning determines how transported features are transformed. This yields architectures suited to multi-physics coupling, irregular geometries, and discretization transfer, with existing neural operators arising as special cases. Empirically, TNOs and HTNOs perform strongly across irregular geometries and higher-rank PDE systems, improving physical consistency. This points toward scientific foundation models that can learn operators on structured spaces, unifying geometry, physics, and computation.

Limitations and future work. Our work leaves ample room for future work: scaling to dynamic, adaptive, and very large domains; developing approximation, stability, and convergence theory, clarifying the role of harmonic and sheaf-based channels; and extending TNOs to multiscale systems, evolving manifolds, inverse problems, and gauge- or symmetry-equivariant operator families.

Acknowledgements. The authors are grateful for support from the UK AI Research Resource (AIRR) through grant 0251-4584-0945-1. T. B. acknowledges support from the UKRI Engineering and Physical Sciences Research Council (EPSRC) through the Future Leaders Fellowship [grant number MR/Y018818/1]. L.B. was supported by the UK Royal Society through grant NIF/R1/254128.

Broader impact. Topological Neural Operators (TNOs) aim to bring geometric and topological structure into operator learning by modeling physical quantities on their natural supports, such as vertices, edges, faces, and volumes. We hope this perspective enables more faithful and efficient surrogate models for scientific computing, with applications in physics, engineering, and simulation-driven design. At the same time, TNOs remain learned approximations and do not guarantee certified physical correctness or numerical stability outside the training distribution. Careful validation against trusted numerical solvers is therefore essential, particularly in safety-critical applications. More broadly, this work explores the initial steps of an exciting idea: that data-driven simulation systems may benefit from respecting the shape and structure of the worlds they model.

References

- [1] Benedikt Alkin, Andreas Fürst, Simon Schmid, Lukas Gruber, Markus Holzleitner, and Johannes Brandstetter. Universal physics transformers: A framework for efficiently scaling neural operators. In *Adv. Neural Inform. Process. Syst.*, 2024.
- [2] Benedikt Alkin, Maurits Bleeker, Richard Kurle, Tobias Kronlachner, Reinhard Sonleitner, Matthias Dorfer, and Johannes Brandstetter. AB-UPT: Scaling neural CFD surrogates for high-fidelity automotive aerodynamics simulations via anchored-branched universal physics transformers. *Trans. Mach. Lear. Resea.*, 2025. arXiv:2502.09692.
- [3] Douglas N. Arnold. *Finite Element Exterior Calculus*, volume 93 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2018. doi: 10.1137/1.9781611975543.
- [4] Douglas N. Arnold, Richard S. Falk, and Ragnar Winther. Multigrid in $H(\text{div})$ and $H(\text{curl})$. *Numerische Mathematik*, 85(2):197–217, 2000.
- [5] Douglas N. Arnold, Richard S. Falk, and Ragnar Winther. Finite element exterior calculus, homological techniques, and applications. *Acta Numerica*, 15:1–155, 2006.

- [6] Douglas N. Arnold, Richard S. Falk, and Ragnar Winther. Finite element exterior calculus: from Hodge theory to numerical stability. *Bulletin of the American Mathematical Society*, 47(2):281–354, 2010.
- [7] Michael Aschbacher. Combinatorial cell complexes. In *Progress in Algebraic Combinatorics*, volume 24 of *Advanced Studies in Pure Mathematics*, pages 1–80. Mathematical Society of Japan, 1996. doi: 10.2969/aspm/02410001.
- [8] Kamyar Azizzadenesheli, Nikola Kovachki, Zongyi Li, Miguel Liu-Schiaffini, Jean Kossaifi, and Anima Anandkumar. Neural operators for accelerating scientific simulations and design. *Nature Reviews Physics*, 6(5):320–328, 2024.
- [9] Igor A. Baratta, Joseph P. Dean, Jørgen S. Dokken, Michal Habera, Jack S. Hale, Chris N. Richardson, Marie E. Rognes, Matthew W. Scroggs, Nathan Sime, and Garth N. Wells. DOLFINx: The next generation FEniCS problem solving environment. Zenodo, 2023.
- [10] Tathagata Basak. Combinatorial cell complexes and poincaré duality. *Geometriae Dedicata*, 147(1):357–387, 2010.
- [11] Nathan Bell and Anil N. Hirani. PyDEC: Software and algorithms for discretization of exterior calculus. *ACM Transactions on Mathematical Software*, 39(1):3:1–3:41, 2012. doi: 10.1145/2382585.2382588.
- [12] Kaushik Bhattacharya, Bamdad Hosseini, Nikola B Kovachki, and Andrew M Stuart. Model reduction and neural networks for parametric PDEs. *The SMAI Journal of Computational Mathematics*, 7:121–157, 2021.
- [13] Ginestra Bianconi. The topological Dirac equation of networks and simplicial complexes. *Journal of Physics: Complexity*, 2(3):035022, 2021.
- [14] Cristian Bodnar, Fabrizio Frasca, Nina Otter, Yuguang Wang, Pietro Liò, Guido Montúfar, and Michael Bronstein. Weisfeiler and lehman go cellular: CW networks. In *Adv. Neural Inform. Process. Syst.*, 2021.
- [15] Cristian Bodnar, Fabrizio Frasca, Yuguang Wang, Nina Otter, Guido Montúfar, Pietro Liò, and Michael Bronstein. Weisfeiler and lehman go topological: Message passing simplicial networks. In *Int. Conf. Mach. Lear.*, 2021.
- [16] Cristian Bodnar, Francesco Di Giovanni, Benjamin Chamberlain, Pietro Lio, and Michael Bronstein. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. *Advances in Neural Information Processing Systems*, 35:18527–18541, 2022.
- [17] Alain Bossavit. Whitney forms: A class of finite elements for three-dimensional computations in electromagnetism. *IEEE Proceedings A (Physical Science, Measurement and Instrumentation, Management and Education, Reviews)*, 135(8):493–500, 1988.
- [18] Alain Bossavit. *Computational Electromagnetism: Variational Formulations, Complementarity, Edge Elements*. Academic Press, 1998.
- [19] Nicolas Boullé and Alex Townsend. A mathematical guide to operator learning. In *Handbook of Numerical Analysis*, volume 25, pages 83–125. Elsevier, 2024.
- [20] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- [21] James H. Bramble, Joseph E. Pasciak, and Jinchao Xu. The analysis of multigrid algorithms with nonnested spaces or noninherited quadratic forms. *Mathematics of Computation*, 56(193): 1–34, 1991.
- [22] William L. Briggs, Van Emden Henson, and Steve F. McCormick. *A Multigrid Tutorial*. SIAM, Philadelphia, second edition, 2000. doi: 10.1137/1.9780898719505.

- [23] Andrey Bryutkin, Jiahao Huang, Zhongying Deng, Guang Yang, Carola-Bibiane Schönlieb, and Angelica I Aviles-Rivero. Hamlet: Graph transformer neural operator for partial differential equations. In *International Conference on Machine Learning*, pages 4624–4641. PMLR, 2024.
- [24] Lucille Calmon, Michael T. Schaub, and Ginestra Bianconi. Higher-order signal processing with the Dirac operator. In *56th Asilomar Conference on Signals, Systems, and Computers*, pages 925–929. IEEE, 2022. doi: 10.1109/IEEECONF56349.2022.10052062.
- [25] Edoardo Calvello, Nikola B Kovachki, Matthew E Levine, and Andrew M Stuart. Continuum attention for neural operators. *Journal of Machine Learning Research*, 26(300):1–52, 2025.
- [26] Gengxiang Chen, Xu Liu, Qinglu Meng, Lu Chen, Changqing Liu, and Yingguang Li. Learning neural operators on riemannian manifolds. *National Science Open*, 3(6):20240001, 2024.
- [27] Chaoran Cheng and Jian Peng. Equivariant neural operator learning with graphon convolution. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, pages 61960–61984, 2023.
- [28] Pengcheng Cheng. Gauge-equivariant intrinsic neural operators for geometry-consistent learning of elliptic pde maps. *arXiv preprint arXiv:2603.14734*, 2026.
- [29] Jae Choi, Yuzhou Chen, Huikyo Lee, Hyun Kim, and Yulia R. Gel. SNN-PDE: Learning dynamic PDEs from data with simplicial neural networks. In *AAAI*, pages 11561–11569, 2024.
- [30] Keenan Crane. Discrete differential geometry: An applied introduction. <https://www.cs.cmu.edu/~kmc Crane/Projects/DDG/paper.pdf>, 2018. Lecture notes, Carnegie Mellon University.
- [31] Keenan Crane, Fernando de Goes, Mathieu Desbrun, and Peter Schröder. Digital geometry processing with discrete exterior calculus. In *ACM SIGGRAPH 2013 Courses*, pages 1–126. 2013.
- [32] Mathieu Desbrun, Anil N Hirani, Melvin Leok, and Jerrold E Marsden. Discrete exterior calculus. *arXiv preprint math/0508341*, 2005.
- [33] Mathieu Desbrun, Eva Kanso, and Yiyang Tong. Discrete differential forms for computational modeling. In *ACM SIGGRAPH 2006 Courses*, pages 39–54. 2006.
- [34] Somdatta Goswami, Aniruddha Bora, Yue Yu, and George Em Karniadakis. Physics-informed deep neural operator networks. In *Machine learning in modeling and simulation: methods and applications*, pages 219–254. Springer, 2023.
- [35] Mustafa Hajij, Ghada Zamzmi, Theodore Papamarkou, Aldo Guzmán-Sáenz, Tolga Birdal, and Michael T. Schaub. Combinatorial complexes: bridging the gap between cell complexes and hypergraphs. In *Asilomar Conference on Signals, Systems, and Computers*, pages 799–803. IEEE, 2023.
- [36] Mustafa Hajij, Ghada Zamzmi, Theodore Papamarkou, Nina Miolane, Aldo Guzmán-Sáenz, Karthikeyan Natesan Ramamurthy, Tolga Birdal, Tamal K. Dey, Soham Mukherjee, Shreyas N. Samaga, Neal Livesay, Robin Walters, Paul Rosen, and Michael T. Schaub. Topological deep learning: Going beyond graph data. *arXiv preprint arXiv:2206.00606*, 2023.
- [37] Mustafa Hajij, Lennart Bastian, Sarah Osentoski, Hardik Kabaria, John L Davenport, Sheik Dawood, Balaji Cherukuri, Joseph G Kocheemoolayil, Nastaran Shahmansouri, Adrian Lew, Theodore Papamarkou, and Tolga Birdal. Copesheaf topological neural networks: A generalized deep learning framework. In *Adv. Neural Inform. Process. Syst.*, 2025.
- [38] Jakob Hansen and Thomas Gebhart. Sheaf neural networks. In *NeurIPS 2020 Workshop on Topological Data Analysis and Beyond*, 2020. arXiv:2012.06333.
- [39] Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, and Jun Zhu. Gnot: A general neural operator transformer for operator learning. In *International conference on machine learning*, pages 12556–12569. PMLR, 2023.

- [40] Maximilian Herde, Bogdan Raonić, Tobias Rohner, Roger Käppeli, Roberto Molinaro, Emmanuel de Bézenac, and Siddhartha Mishra. Poseidon: Efficient foundation models for PDEs. *Advances in Neural Information Processing Systems*, 37:72525–72624, 2024.
- [41] Ralf Hiptmair. Multigrid method for Maxwell’s equations. *SIAM Journal on Numerical Analysis*, 36(1):204–225, 1998.
- [42] Ralf Hiptmair. Finite elements in computational electromagnetism. *Acta Numerica*, 11:237–339, 2002.
- [43] Ralf Hiptmair and Jinchao Xu. Nodal auxiliary space preconditioning in $H(\text{curl})$ and $H(\text{div})$ spaces. *SIAM Journal on Numerical Analysis*, 45(6):2483–2509, 2007.
- [44] Anil Nirmal Hirani. *Discrete Exterior Calculus*. PhD thesis, California Institute of Technology, 2003.
- [45] James M Hyman, Mikhail Shashkov, and Stanly Steinberg. The numerical solution of diffusion problems in strongly heterogeneous non-isotropic materials. *Journal of Computational Physics*, 132(1):130–148, 1997.
- [46] Anran Jiao, Haiyang He, Rishikesh Ranade, Jay Pathak, and Lu Lu. One-shot learning for solution operators of partial differential equations. *Nature Communications*, 16(1):8386, 2025.
- [47] Nikola B Kovachki, Samuel Lanthaler, and Andrew M Stuart. Operator learning: Algorithms and analysis. *Handbook of Numerical Analysis*, 25:419–467, 2024.
- [48] Nikolas Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *J. Mach. Learn. Res.*, 24(89):1–97, 2023.
- [49] Samuel Leventhal, Attila Gyulassy, Valerio Pascucci, and Mark Heimann. Modeling hierarchical topological structure in scientific images with graph neural networks. In *2023 IEEE International Conference on Image Processing (ICIP)*, pages 2995–2999. IEEE, 2023.
- [50] Zijie Li, Kazem Meidani, and Amir Barati Farimani. Transformer for partial differential equations’ operator learning. *Transactions on Machine Learning Research*, 2023.
- [51] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. In *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*, 2020. arXiv:2003.03485.
- [52] Zongyi Li, Nikolas Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *ICLR*, 2021.
- [53] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for PDEs on general geometries. *J. Mach. Learn. Res.*, 24(388):1–26, 2023.
- [54] Zongyi Li, Nikola Kovachki, Chris Choy, Boyi Li, Jean Kossaifi, Shourya Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, and Anima Anandkumar. Geometry-informed neural operator for large-scale 3D PDEs. In *Adv. Neural Inform. Process. Syst.*, 2023.
- [55] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/IMS Journal of Data Science*, 1(3):1–27, 2024.
- [56] Yunfeng Liao, Yangxin Wu, and Xiucheng Li. Boundary-value PDEs meet higher-order differential topology-aware GNNs. In *Advances in Neural Information Processing Systems*, 2025.

- [57] Levi E. Lingsch, Mike Yan Michelis, Emmanuel De Bezenac, Sirani M. Perera, Robert K. Katzschmann, and Siddhartha Mishra. Beyond regular grids: Fourier-based neural operators on arbitrary domains. In *Int. Conf. Mach. Lear.*, 2024.
- [58] Konstantin Lipnikov, Gianmarco Manzini, and Mikhail Shashkov. Mimetic finite difference method. *Journal of Computational Physics*, 257:1163–1227, 2014.
- [59] Ning Liu, Siavash Jafarzadeh, and Yue Yu. Domain agnostic fourier neural operators. *Advances in neural information processing systems*, 36:47438–47450, 2023.
- [60] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.
- [61] Mark Meyer, Mathieu Desbrun, Peter Schröder, and Alan H Barr. Discrete differential-geometry operators for triangulated 2-manifolds. In *Visualization and mathematics III*, pages 35–57. Springer, 2003.
- [62] Peter Monk. A finite element method for approximating the time-harmonic Maxwell equations. *Numerische Mathematik*, 63(2):243–262, 1992. doi: 10.1007/BF01385860.
- [63] Peter Monk. *Finite Element Methods for Maxwell’s Equations*. Oxford University Press, Oxford, 2003.
- [64] Sepehr Mousavi, Shizheng Wen, Levi Lingsch, Maximilian Herde, Bogdan Raonic, and Siddhartha Mishra. Rigno: A graph-based framework for robust and accurate operator learning for pdes on arbitrary domains. In *Adv. Neural Inform. Process. Syst.*, 2025.
- [65] Jean-Claude Nédélec. Mixed finite elements in $\mathbb{R}^3$. *Numerische Mathematik*, 35(3):315–341, 1980.
- [66] Fabian Paischer, Leo Cotteleer, Yann Dreze, Richard Kurle, Dylan Rubini, Maurits Bleeker, Tobias Kronlachner, and Johannes Brandstetter. Going with the speed of sound: Pushing neural surrogates into highly-turbulent transonic regimes. In *NeurIPS 2025 Workshop on Machine Learning and the Physical Sciences*, 2025. arXiv:2511.21474.
- [67] Theodore Papamarkou, Tolga Birdal, Michael M. Bronstein, Gunnar E. Carlsson, Justin Curry, Yue Gao, Mustafa Hajj, Roland Kwitt, Pietro Liò, Paolo Di Lorenzo, Vasileios Maroulas, Nina Miolane, Farzana Nasrin, Karthikeyan Natesan Ramamurthy, Bastian Rieck, Simone Scardapane, Michael T. Schaub, Petar Veličković, Bei Wang, Yusu Wang, Guowei Wei, and Ghada Zamzmi. Position: Topological deep learning is the new frontier for relational learning. In *Int. Conf. Mach. Lear.*, volume 235 of *Proceedings of Machine Learning Research*, pages 39529–39555, 2024.
- [68] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [69] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. In *ICLR*, 2021.
- [70] Lenka Ptáčeková and Luiz Velho. A simple and complete discrete exterior calculus on general polygonal meshes. *Computer Aided Geometric Design*, 88:102002, 2021.
- [71] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *CVPR*, 2017.
- [72] Blaine Quackenbush and Paul J Atzberger. Geometric neural operators (gnps) for data-driven deep learning in non-euclidean settings. *Machine Learning: Science and Technology*, 5(4): 045033, 2024.
- [73] Bogdan Raonic, Roberto Molinaro, Tim De Ryck, Tobias Rohner, Francesca Bartolucci, Rima Alaifari, Siddhartha Mishra, and Emmanuel de Bézenac. Convolutional neural operators for robust and accurate learning of PDEs. In *Adv. Neural Inform. Process. Syst.*, 2023.

- [74] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics, Philadelphia, 2 edition, 2003.
- [75] Maxime Savoy. Combinatorial cell complexes: Duality, reconstruction and causal cobordisms. *arXiv preprint arXiv:2201.12846*, 2022.
- [76] Michael T Schaub, Austin R Benson, Paul Horn, Gabor Lippner, and Ali Jadbabaie. Random walks on simplicial complexes and the normalized hodge 1-laplacian. *SIAM Review*, 62(2): 353–391, 2020.
- [77] Dmitriy Smirnov and Justin Solomon. HodgeNet: Learning spectral geometry on triangle meshes. *ACM Transactions on Graphics (SIGGRAPH)*, 40(4), 2021.
- [78] Yiyong Tong, Santiago Lombeyda, Anil N. Hirani, and Mathieu Desbrun. Discrete multiscale vector field decomposition. *ACM Transactions on Graphics*, 22(3):445–452, 2003. doi: 10.1145/882262.882290. (Proc. SIGGRAPH 2003).
- [79] Alasdair Tran, Alexander Mathews, Lexing Xie, and Cheng Soon Ong. Factorized fourier neural operators. In *The Eleventh International Conference on Learning Representations*, 2023.
- [80] Nathaniel Trask, Andy Huang, and Xiaozhe Hu. Enforcing exact physics in scientific machine learning: A data-driven exterior calculus on graphs. *Journal of Computational Physics*, 456: 110969, 2022.
- [81] Tapas Tripura and Souvik Chakraborty. Wavelet neural operator for solving parametric partial differential equations in computational mechanics problems. *Computer Methods in Applied Mechanics and Engineering*, 404:115783, 2023.
- [82] Ulrich Trottenberg, Cornelius W. Oosterlee, and Anton Schüller. *Multigrid*. Academic Press, London, 2001.
- [83] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Adv. Neural Inform. Process. Syst.*, 2017.
- [84] Francesco Viganò, Tolga Birdal, Michael T Schaub, and Mauricio Barahona. Root-to-leaf path random walks, normalized hodge laplacians, and cheeger inequalities on simplicial complexes. *arXiv preprint arXiv:2604.27241*, 2026.
- [85] Hrishikesh Viswanath, Md Ashiqur Rahman, Abhijeet Vyas, Andrey Shor, Beatriz Medeiros, Stephanie Hernandez, Suhas Eswarappa Prameela, and Aniket Bera. Neural operator: Is data all you need to model the world? an insight into the paradigm of data-driven scientific ML. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026. doi: 10.1109/TPAMI.2026.3682604.
- [86] Shizheng Wen, Arsh Kumbhat, Levi Lingsch, Sepehr Mousavi, Yizhou Zhao, Praveen Chandrashekar, and Siddhartha Mishra. Geometry aware operator transformer as an efficient and accurate neural surrogate for PDEs on arbitrary domains. In *Adv. Neural Inform. Process. Syst.*, 2025. arXiv:2505.18781.
- [87] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for PDEs on general geometries. In *Int. Conf. Mach. Lear.*, 2024.
- [88] Minglang Yin, Nicolas Charon, Ryan Brody, Lu Lu, Natalia Trayanova, and Mauro Maggioni. A scalable framework for learning the geometry-dependent solution operators of partial differential equations. *Nature Computational Science*, 4(12):928–940, 2024.
- [89] Seungwoo Yoo, Kyeongmin Yeo, Jisung Hwang, and Minhyuk Sung. Neural Green’s functions. In *Adv. Neural Inform. Process. Syst.*, 2025.
- [90] Weiheng Zhong and Hadi Meidani. Physics-informed geometry-aware neural operator. *Computer Methods in Applied Mechanics and Engineering*, 434:117540, 2025.

Topological Neural Operators

Appendix

Table of Contents

A Background	16
B PDE Templates as Typed Cochain Systems	25
C Proofs	28
D TNOs, Hodge-Compatible Smoothing, and the Multigrid Correspondence	31
E Implementation Details	36
F Experimental Details	38
G Extended Experimental Results	40
H Related Work	42

A Background

A.1 Combinatorial Complexes

Combinatorial complexes generalize simplicial and cell complexes as well as hypergraphs and act as a unifying topological structure:

Definition A.1 (Combinatorial complex [36, 35]). A combinatorial complex (CC) is a triple $(S, \mathcal{X}, \text{rk})$ consisting of a set S of entities, a subset $\mathcal{X} \subseteq \mathcal{P}(S) \setminus \{\emptyset\}$ of cells, and a rank function $\text{rk} : \mathcal{X} \rightarrow \mathbb{Z}_{\geq 0}$ satisfying $\text{rk}(\{v\}) = 0$ for all $v \in S$ and $\text{rk}(x) \leq \text{rk}(y)$ whenever $x \subseteq y$.

Lifting an RCC K to a CC $\tilde{K}$ retains the cochains of K but may carry additional cells—higher-order groupings or augmented neighborhoods—to enrich the information flow. The lift does not change the target operator $\mathcal{G}$ or the input / output spaces $\mathcal{X}(K), \mathcal{Y}(K)$; it only enlarges the domain on which the layer acts. See Fig. 5.

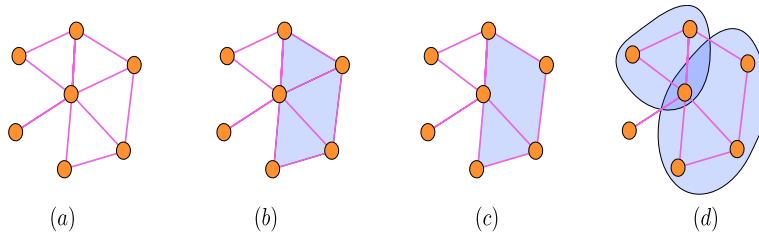


Figure 5: Progression of topological domains with increasing modeling flexibility. **(a)** A graph represents only vertices and pairwise edges. **(b)** A simplicial complex adds higher-order simplices, but each simplex is determined by its lower-dimensional faces. **(c)** A cell complex allows more general cells, such as polygonal regions, whose boundaries need not be simplices. **(d)** A combinatorial complex further relaxes the construction by allowing independently specified, possibly overlapping higher-order relations. Each step increases the flexibility of the domain on which cochains and topological neural operators can be defined.

A.2 Discrete Differential Forms and Physical Cochains

We recall the discrete differential-form viewpoint used throughout the paper [33]. The point is not to introduce differential forms for their own sake, but to explain why physical fields should not all be represented as vertex signals. Many physical quantities are not point samples. A potential is naturally measured at points, a circulation along curves, a flux through surfaces, and a density over volumes. Discrete exterior calculus (DEC) keeps this measurement type in the discrete model.

Throughout, K denotes an oriented regular cell complex. We write K_k for the set of k -cells: vertices for $k = 0$, edges for $k = 1$, faces for $k = 2$, and volumes for $k = 3$. Examples of regular cell complexes are shown in Fig. 6.

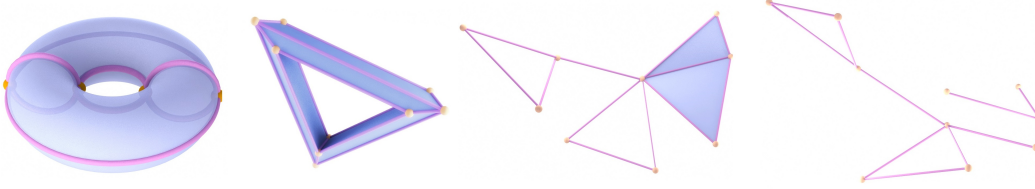


Figure 6: Four examples of regular cell complexes.

A *discrete k -form*, or *k -cochain*, is a field supported on oriented k -cells:

$$C^k(K; \mathbb{R}^d) = \{u^k : K_k \rightarrow \mathbb{R}^d\} \cong \mathbb{R}^{|K_k| \times d}.$$

Thus 0-cochains live on vertices, 1-cochains on edges, 2-cochains on faces, and 3-cochains on volumes. The degree k is therefore not merely an index. It records the geometric type of the quantity.

The useful intuition is that cochains are *cellular measurements*. A 0-cochain stores point measurements. A 1-cochain stores oriented line measurements. A 2-cochain stores oriented surface measurements. A 3-cochain stores volume measurements. This is the discrete analogue of the integral viewpoint in exterior calculus: one does not approximate every physical field by point values; one stores each quantity on the cells over which it is naturally measured.

This distinction matters. A flux through a face is not the same kind of object as a scalar value at a vertex. A circulation along an edge is not merely another node feature. If such fields are collapsed to vertices, their orientation, incidence, and conservation relations become hidden or must be relearned from data. DEC avoids this collapse by making the support dimension part of the representation.

Chains, cochains, and discrete integration. Let K be an oriented cell complex and let K_k denote its set of oriented k -cells. A k -chain is a finite formal linear combination of k -cells,

$$c = \sum_{\sigma \in K_k} c_\sigma \sigma, \quad c_\sigma \in \mathbb{R}.$$

The coefficients c_σ record how the cells are assembled. A positive coefficient uses the chosen orientation of σ , while a negative coefficient uses the opposite orientation. Thus a chain is a discrete geometric domain: a 0-chain is a signed collection of vertices, a 1-chain is an oriented path made of edges, a 2-chain is an oriented surface patch made of faces, and so on.

In this paragraph we take cochains to be scalar-valued, since this is the setting in which the integration pairing and Stokes' theorem are most directly stated. A k -cochain is a linear functional on k -chains. Equivalently, it is specified by assigning one scalar to each oriented k -cell,

$$u^k : K_k \rightarrow \mathbb{R}.$$

Its value on a chain $c = \sum_{\sigma \in K_k} c_\sigma \sigma$ is defined by linearity:

$$\langle u^k, c \rangle = \sum_{\sigma \in K_k} c_\sigma u^k(\sigma).$$

This pairing is the discrete analogue of integration. If ω^k is a smooth k -form and R is a k -dimensional oriented region, then the continuum measurement is

$$\int_R \omega^k.$$

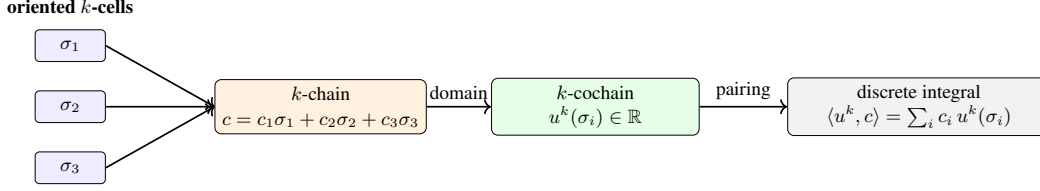


Figure 7: Chains and cochains as discrete integration. A k -chain is a signed combination of oriented k -cells and represents a discrete k -dimensional domain. A k -cochain assigns measurements to oriented k -cells. Their pairing sums the cochain values over the chain with the chain coefficients, giving the discrete analogue of integrating a k -form over a k -dimensional region.

In the discrete setting, the region R is replaced by a k -chain c , the form ω^k is replaced by a k -cochain u^k , and the integral is replaced by

$$\int_R \omega^k \rightsquigarrow \langle u^k, c \rangle.$$

This is the sense in which cochains are discrete integrands. A 1-cochain can be summed along an oriented path. A 2-cochain can be summed over an oriented surface patch. A 3-cochain can be summed over a collection of volume cells. The degree k specifies both where the quantity is stored and what kind of discrete region it can be measured on. See Figure 7.

For neural-network features we often use vector-valued cochains $u^k : K_k \rightarrow \mathbb{R}^{d_k}$. These should be understood as d_k scalar cochains stored in parallel, with the same pairing applied componentwise. The scalar case is the one used to state the discrete Stokes relation; the vector-valued case is its channel-wise extension.

This interpretation prepares the definition of the exterior derivative. Once cochains are viewed as discrete integrands, Stokes' theorem becomes a statement about how the measurement of a derivative on a cell is related to the measurement of the original cochain on the oriented boundary of that cell.

From cellular measurements to the discrete exterior derivative.. A cochain stores measurements on cells. The degree records the type of measurement: values on vertices, line measurements on edges, surface measurements on faces, and volume measurements on 3-cells. This is the de Rham convention used throughout the paper: a physical quantity is placed on the cell type over which it is naturally measured.

Table 4: Physical quantities and their natural cochain degrees under the primal de Rham convention. A k -form is discretized as a k -cochain by integration over oriented k -cells.

Physical quantity	Geometric type	Degree k	Primal cell
Temperature, pressure, potential	0-form	0	Vertices
Electric field E , circulation, line flow	1-form	1	Edges
Magnetic flux B , vorticity flux, Darcy flux	2-form	2	Faces
Volumetric charge, mass, source density	3-form	3	Volumes

Table 4 should be read as a statement about measurement, not only about notation. A scalar potential is sampled at points. A circulation or line integral is measured along an oriented curve, and in the discrete complex, this means an edge. A flux is measured through an oriented surface, and in the discrete complex, this means a face. A density or source term is measured over a volume. This is why the cochain degree is part of the field's physical meaning.

The table uses the primal convention. In an n -dimensional domain, the Hodge star identifies primal k -cochains with dual $(n - k)$ -cochains. Consequently, the same physical vector quantity may be stored on different cells depending on whether one uses the primal or dual complex. Unless stated otherwise, this paper uses the primal convention in Table 4.

The question is then how to differentiate such quantities without forgetting where they are measured. The classical answer is that differentiation and boundary measurement are tied together. In one

dimension, the fundamental theorem of calculus says that the integral of a derivative over an interval is determined by the values at the two boundary points:

$$\int_a^b f'(t) dt = f(b) - f(a).$$

If the interval is oriented from a to b , its boundary is the formal signed sum

$$\partial[a, b] = [b] - [a].$$

Thus the derivative measured on the interval is obtained from the original function measured on the oriented boundary. The endpoint at the head of the interval contributes with sign $+1$, and the endpoint at the tail contributes with sign -1 .

The same boundary principle appears in higher dimensions. For a vector field in the plane, Green's theorem relates circulation around the boundary of a region to a curl-type quantity inside the region. In three dimensions, Stokes' theorem relates circulation around the boundary curve of a surface to curl through the surface. Gauss' theorem relates flux through the boundary surface of a volume to divergence inside the volume. Exterior calculus writes these boundary laws as one statement:

$$\int_R d\omega = \int_{\partial R} \omega.$$

Here ω is measured on k -dimensional objects, $d\omega$ is measured on $(k+1)$ -dimensional objects, R is a $(k+1)$ -dimensional region, and ∂R is its oriented boundary. The identity states that the derivative of a quantity over a region is determined by the original quantity on the boundary of that region.

This is the viewpoint DEC keeps. Since a cochain already stores integrated measurements on cells, the discrete derivative is defined by the same boundary relation. A vertex quantity differentiates to an edge quantity by taking signed endpoint differences. An edge quantity differentiates to a face quantity by taking signed circulation around the face boundary. A face quantity differentiates to a volume quantity by taking signed flux through the volume boundary. The common operation is evaluation on an oriented boundary.

On a cell complex, the role of the region R is played by a $(k+1)$ -cell τ , and the role of ∂R is played by the oriented cellular boundary of τ . Let

$$B_{k+1} : C_{k+1}(K) \rightarrow C_k(K)$$

be the oriented boundary matrix. Its columns are indexed by $(k+1)$ -cells and its rows are indexed by k -cells. For $\tau \in K_{k+1}$, the column indexed by τ records

$$\partial\tau = \sum_{\sigma \in K_k} [\tau : \sigma] \sigma, \quad [\tau : \sigma] \in \{-1, 0, +1\}.$$

The coefficient $[\tau : \sigma]$ is nonzero exactly when σ is a boundary face of τ . The sign records orientation: $+1$ if the chosen orientation of σ agrees with the orientation induced from τ , and -1 if it disagrees.

Now let $u^k \in C^k(K)$ be a k -cochain. Its discrete exterior derivative $d^k u^k$ is the $(k+1)$ -cochain obtained by measuring u^k on the oriented boundary of each $(k+1)$ -cell:

$$\langle d^k u^k, \tau \rangle = \langle u^k, \partial\tau \rangle, \quad \tau \in K_{k+1}.$$

In matrix form,

$$d^k : C^k(K) \rightarrow C^{k+1}(K), \quad d^k = B_{k+1}^\top.$$

Equivalently,

$$(d^k u^k)(\tau) = \sum_{\sigma \prec \tau} [\tau : \sigma] u^k(\sigma).$$

Thus d^k takes measurements on k -cells and produces measurements on $(k+1)$ -cells by signed summation over oriented boundaries. Its nonzero pattern comes from incidence, and its signs come from orientation. This is why the discrete exterior derivative is a topological operator.

For TNOs, this is the basic upward cross-rank route. Vertex cochains are coupled to edge cochains, edge cochains to face cochains, and face cochains to volume cochains through the same boundary rule

that appears in the underlying differential equations. The cell complex fixes the route; the learnable part is the channel mixing applied to the routed cochain features.

In low degrees, we obtain the sequence

$$C^0(K) \xrightarrow{d^0} C^1(K) \xrightarrow{d^1} C^2(K) \xrightarrow{d^2} C^3(K).$$

These maps are the cellular analogues of the familiar vector-calculus operators:

$$d^0 = \text{gradient-type map}, \quad d^1 = \text{curl-type map}, \quad d^2 = \text{divergence-type map}.$$

Here “type” means that the maps act between cellular measurement spaces. The map d^0 takes vertex measurements to edge measurements. The map d^1 takes edge measurements to face measurements. The map d^2 takes face measurements to volume measurements.

For $x \in C^0(K)$ and an oriented edge $e = [i, j]$,

$$(d^0 x)([i, j]) = x_j - x_i.$$

This is the fundamental theorem of calculus on a single edge: the measurement of the derivative along the edge is the signed difference of the endpoint values. Hence $d^0 x$ is an edge cochain.

For $y \in C^1(K)$ and an oriented face $f \in K_2$,

$$(d^1 y)(f) = \sum_{e \prec f} [f : e] y(e).$$

The value on the face is the signed circulation of the edge measurements around its boundary. This is the cellular version of Green’s/Stokes’ theorem. For an oriented triangle $[i, j, k]$, the induced boundary is

$$\partial[i, j, k] = [j, k] - [i, k] + [i, j].$$

With stored edge orientations $[i, j]$, $[i, k]$, and $[j, k]$, this gives

$$(d^1 y)([i, j, k]) = y_{ij} - y_{ik} + y_{jk}.$$

If the edge is stored as $[k, i]$ rather than $[i, k]$, the same circulation is written as

$$(d^1 y)([i, j, k]) = y_{ij} + y_{jk} + y_{ki}.$$

The two expressions represent the same oriented boundary measurement. The signs make the result independent of the bookkeeping choice for edge orientations.

For $z \in C^2(K)$ and an oriented volume cell $c \in K_3$,

$$(d^2 z)(c) = \sum_{f \prec c} [c : f] z(f).$$

The value on the volume is the signed accumulation of the face measurements on its boundary. This is the cellular version of Gauss’ theorem: it measures net boundary flux.

The following example makes the construction explicit. Consider the oriented complex in Fig. 8 with

$$K_0 = ([1], [2], [3], [4], [5]),$$

$$K_1 = ([1, 2], [1, 3], [2, 3], [2, 4], [3, 4], [4, 5]), \quad K_2 = ([1, 2, 3], [2, 3, 4]).$$

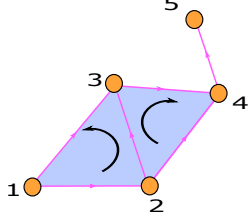
The matrix d^0 has one row per edge and one column per vertex. The row indexed by $[i, j]$ has a -1 in column i , a $+1$ in column j , and zeros elsewhere. Multiplying d^0 by a vertex cochain therefore computes all oriented edge differences at once.

The matrix d^1 has one row per face and one column per edge. Each row is the oriented boundary of one face written in the chosen edge basis. For the first face,

$$\partial[1, 2, 3] = [2, 3] - [1, 3] + [1, 2],$$

so the first row of d^1 , using the edge ordering above, is

$$[1, -1, 1, 0, 0, 0].$$



$$d^0 = B_1^\top : C^0(K) \rightarrow C^1(K), \quad d^1 = B_2^\top : C^1(K) \rightarrow C^2(K)$$

$$d^0 = \begin{bmatrix} -1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix}, \quad d^1 = \begin{bmatrix} 1 & -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 1 & 0 \end{bmatrix}.$$

Figure 8: Coboundary matrices for the oriented complex. The matrix d^0 maps vertex cochains to edge cochains: each row computes one oriented difference, e.g. $[-1, 1, 0, 0, 0]$ gives $x_2 - x_1$ on $[1, 2]$. The matrix d^1 maps edge cochains to face cochains: each row computes one signed circulation. For the face $[1, 2, 3]$, the boundary relation $\partial[1, 2, 3] = [2, 3] - [1, 3] + [1, 2]$ gives $(d^1 y)([1, 2, 3]) = y_{12} - y_{13} + y_{23}$; similarly, $(d^1 y)([2, 3, 4]) = y_{23} - y_{24} + y_{34}$. Hence these matrices encode oriented incidence, not just adjacency.

For the second face,

$$\partial[2, 3, 4] = [3, 4] - [2, 4] + [2, 3],$$

so the second row is

$$[0, 0, 1, -1, 1, 0].$$

Multiplying d^1 by an edge cochain y returns one circulation value per filled face.

The example shows the general rule. The operator d^k evaluates a k -cochain on the oriented boundary of each $(k+1)$ -cell. This is the discrete Stokes map. It is also the basic upward route used by TNO layers for cross-rank coupling.

Boundaries cancel in pairs. The boundary-based definition of d^k has an immediate consequence: applying two such maps in a row gives zero. Algebraically,

$$B_k B_{k+1} = 0, \quad \text{equivalently} \quad d^{k+1} d^k = 0.$$

The first identity says that the boundary of a cell has no remaining boundary. The second identity is the corresponding statement on cochains:

$$C^k(K) \xrightarrow{d^k} C^{k+1}(K) \xrightarrow{d^{k+1}} C^{k+2}(K), \quad d^{k+1} d^k = 0.$$

The reason is purely local. Take a $(k+2)$ -cell and look at the k -cells that appear after taking its boundary twice. Each such k -cell appears exactly twice, once from each adjacent $(k+1)$ -face, and the two induced orientations are opposite. The two contributions cancel. Thus, the identity is not a limiting statement and does not depend on mesh refinement, training, or numerical accuracy. It holds exactly on the oriented cell complex.

In low degrees, this gives the familiar compatibility laws

$$d^1 d^0 = 0 \quad \Longleftrightarrow \quad \text{curl}(\text{grad}) = 0,$$

and

$$d^2 d^1 = 0 \quad \Longleftrightarrow \quad \text{div}(\text{curl}) = 0.$$

The first says that a field built from vertex differences has zero signed circulation around every filled face. Around a triangle, the edge differences telescope:

$$(x_j - x_i) + (x_k - x_j) + (x_i - x_k) = 0.$$

The second says that a face field built from edge circulations has zero signed accumulation around every volume. Each interior edge contribution appears twice with opposite signs when the face circulations around the volume are summed.

Topology fixes incidence; geometry weights it. The matrices B_k and $d^k = B_{k+1}^\top$ know which cells touch and how their orientations agree. They do not know lengths, areas, volumes, angles, or material coefficients. Geometry enters through Hodge-star, or mass, matrices

$$M_k \in \mathbb{R}^{|K_k| \times |K_k|}, \quad M_k \succ 0,$$

which define inner products on cochains:

$$\langle u^k, v^k \rangle_{M_k} = (u^k)^\top M_k v^k.$$

The matrix M_k stores metric and material information at degree k : edge lengths, face areas, cell volumes, anisotropic weights, permittivity, permeability, conductivity, or other coefficient fields. In primal-dual DEC, M_k can be interpreted as the map relating a primal k -cochain to a dual $(n - k)$ -cochain.

The codifferential is the metric adjoint of d . The codifferential maps downward in degree,

$$\delta^k : C^k(K) \rightarrow C^{k-1}(K),$$

and is defined as the adjoint of d^{k-1} with respect to the Hodge inner products:

$$\langle d^{k-1} u^{k-1}, v^k \rangle_{M_k} = \langle u^{k-1}, \delta^k v^k \rangle_{M_{k-1}}.$$

In matrix form, up to the global sign convention chosen for the Hodge star,

$$\delta^k = M_{k-1}^{-1} B_k M_k.$$

Thus d moves cochains upward by taking signed boundary sums, while δ moves cochains downward by the metric adjoint operation. This is why δ is divergence-like: it measures how a k -cell quantity accumulates onto incident $(k - 1)$ -cells after applying the appropriate geometric weights.

Concretely, at $k = 2$ the codifferential $\delta^2 = M_1^{-1} B_2 M_2 = M_1^{-1} (d^1)^\top M_2$ sends a face cochain B to the edge cochain

$$(\delta^2 B)_e = (M_1^{-1})_{ee} \sum_{f \succ e} [f : e] (M_2)_{ff} B_f, \quad (16)$$

illustrated in Fig. 9.

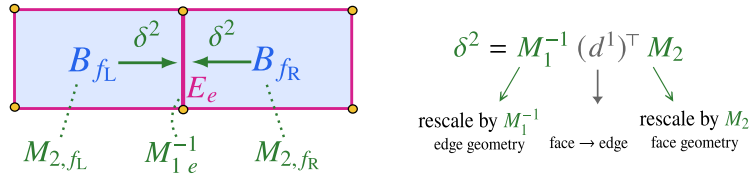


Figure 9: Codifferential as face-to-edge transport (Eq. (16)): neighboring faces $f_L, f_R \in K_2$ sharing edge $e \in K_1$ contribute via the incidence sign $[f : e]$, weighted by the Hodge factors M_2 and M_1^{-1} .

This clarifies the role of the DEC maps inside a TNO. The incidence matrices B_k define the cellular routes along which information can be transported: they specify which cells are incident, how ranks are coupled, and how orientations enter the coupling. The exterior derivative $d^k = B_{k+1}^\top$ moves a k -cochain upward to $(k + 1)$ -cells by signed aggregation over boundaries. On the other hand, the codifferential $\delta^k = M_{k-1}^{-1} B_k M_k$

moves a k -cochain downward to $(k - 1)$ -cells by the metric-weighted adjoint route. Thus d^k and δ^k are not generic message-passing maps: they are structured operators for moving, coupling, and aggregating cochain features across adjacent ranks. The Hodge matrices M_k determine how geometry and material coefficients weight these transports.

The Hodge Laplacian has two same-rank channels. The k -Hodge Laplacian decomposes as

$$\Delta_k = \underbrace{\delta^{k+1} d^k}_{\Delta_k^\uparrow} + \underbrace{d^{k-1} \delta^k}_{\Delta_k^\downarrow} : C^k(K) \rightarrow C^k(K).$$

Both terms start and end in $C^k(K)$, but they route the signal through different adjacent ranks. The upper term sends a k -cochain through $(k + 1)$ -cells and back, while the lower term sends it through $(k - 1)$ -cells and back. See Figure 11. Thus Δ_k is not only a same-rank operator; it records which neighboring dimension mediates the interaction.

For 0-cochains, the lower channel is absent:

$$\Delta_0 = \delta^1 d^0,$$

$$\begin{array}{ccccc}
\Delta_{k-1} & & \Delta_k & & \Delta_{k+1} \\
\curvearrowright & & \curvearrowright & & \curvearrowright \\
C^{k-1}(K) & \xrightarrow{d^{k-1}} & C^k(K) & \xrightarrow{d^k} & C^{k+1}(K) \\
& \xleftarrow{\delta^k} & & \xleftarrow{\delta^{k+1}} & \\
\Delta_k = \underbrace{\delta^{k+1}d^k}_{\Delta_k^\uparrow} + \underbrace{d^{k-1}\delta^k}_{\Delta_k^\downarrow}.
\end{array}$$

Figure 10: Upper and lower Hodge–Laplacian channels around $C^k(K)$. The differential d raises degree and moves rightward, while the codifferential δ lowers degree and moves leftward. The upper channel $\delta^{k+1}d^k$ propagates k -cochains through $(k+1)$ -cells; the lower channel $d^{k-1}\delta^k$ propagates them through $(k-1)$ -cells. The self-loops indicate same-rank Hodge Laplacian propagation on each cochain space.

which is the usual graph or grid Laplacian on vertex fields. For 1-cochains, both channels are present:

$$\Delta_1 = \underbrace{\delta^2 d^1}_{\text{face-mediated}} + \underbrace{d^0 \delta^1}_{\text{vertex-mediated}}.$$

The first term sends edge values around faces and back to edges, coupling edges that bound common 2-cells. This is the circulation, or curl-type, route. The second term sends edge values to vertices and back to edges, coupling edges that meet at common 0-cells. This is the accumulation, or divergence-type, route. See Figure 11. Hence an edge field has two distinct edge-to-edge propagation mechanisms: one mediated by faces and one mediated by vertices. Treating edge fields as ordinary graph features collapses this distinction into a single adjacency-based route.

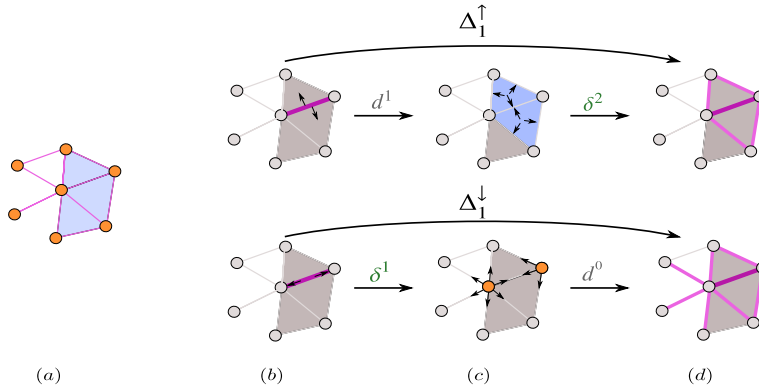


Figure 11: Upper and lower Hodge–Laplacian channels on edge cochains. Starting from an edge-supported signal $u \in C^1(K)$, the upper channel first applies $d^1 : C^1(K) \rightarrow C^2(K)$, which computes signed circulation around each face, and then applies $\delta^2 : C^2(K) \rightarrow C^1(K)$, which returns this face quantity to incident edges through the metric-weighted adjoint route. This gives $\Delta_1^\uparrow u = \delta^2 d^1 u$, a face-mediated edge-to-edge coupling. The lower channel first applies $\delta^1 : C^1(K) \rightarrow C^0(K)$, which measures the metric-weighted accumulation of the edge field at vertices, and then applies $d^0 : C^0(K) \rightarrow C^1(K)$, which sends the resulting vertex potential back to edges by oriented differences. This gives $\Delta_1^\downarrow u = d^0 \delta^1 u$, a vertex-mediated edge-to-edge coupling. Thus $\Delta_1^\uparrow$ and $\Delta_1^\downarrow$ are two distinct same-rank propagation mechanisms: one couples edges through shared faces, the other through shared vertices. This distinction is the degree-1 DEC analogue of separating curl-type and divergence-type effects, and is precisely what is collapsed when edge fields are treated as ordinary graph features.

Hodge decomposition separates physical modes.. With the Hodge inner product, the k -cochain space decomposes as

$$C^k(K; \mathbb{R}) = \underbrace{\text{im}(d^{k-1})}_{\text{exact}} \oplus \underbrace{\ker(\Delta_k)}_{\text{harmonic}} \oplus \underbrace{\text{im}(\delta^{k+1})}_{\text{coexact}}.$$

Exact cochains come from lower-degree potentials. Coexact cochains come from higher-degree flux or circulation potentials. Harmonic cochains satisfy

$$d^k h = 0, \quad \delta^k h = 0, \quad \Delta_k h = 0,$$

and encode global topological degrees of freedom. In the scalar case,

$$\dim \ker(\Delta_k) = \beta_k,$$

the k -th Betti number. Thus harmonic components are not local noise: they are the cochain modes supported by the topology of the domain.

This is one of the main reasons DEC is useful for operator learning. Many physical fields naturally live in one of these components. Conservative fields are exact. Divergence-free fields are coexact, up to harmonic components. Topological modes, such as circulation around non-contractible cycles, appear in the harmonic subspace. A model that does not expose these channels must infer them indirectly from data.

A.3 Continuum Physics and the mPDE Formulation

The governing equations of continuum physics describe how quantities living on different geometric dimensions interact. The electric field $\mathbf{E}$, integrated along oriented edges, and the magnetic flux $\mathbf{B}$, integrated over oriented faces, are therefore naturally a 1-cochain and a 2-cochain. Faraday’s law, $\partial_t B = -d^1 E$, couples them through the exterior derivative d^1 , which maps edge-valued data to face-valued data by accumulating signed values around each face. In physical terms, the circulation of $\mathbf{E}$ around a face boundary equals the negative rate of change of the magnetic flux through that face. Likewise, the incompressibility constraint $d^1 v = 0$ expresses divergence-freeness as a structural property rather than a penalty to enforce. More broadly, many physical laws are organized by the identity $d^{k+1} \circ d^k = 0$, which encodes, in a single algebraic statement, facts such as the absence of magnetic monopoles, the exactness of conservative fields, and the compatibility of flows. All are consequences of the same geometric principle: the boundary of a boundary is empty. The natural discrete setting is a cell complex K — vertices, edges, faces, and higher-dimensional cells with orientations — the minimal structure on which Stokes’ theorem holds exactly. The theorem forces $B_k B_{k+1} = 0$, from which the exterior derivative $d^k = B_{k+1}^\top$, the codifferentials δ^k , and the Hodge Laplacians Δ_k all follow, forming the discrete exterior calculus on K — the language in which physically coupled systems can be written and, therefore, the correct domain for learning their solution operators. On K , the governing laws of a broad class of physical systems define an operator

$$G : \bigoplus_i C^{k_i}(K; \mathbb{R}^{d_i}) \rightarrow \bigoplus_j C^{\ell_j}(K; \mathbb{R}^{r_j}),$$

implicitly characterized by

$$\mathcal{F}_k(\sigma, u^k, d^{k-1}u^{k-1}, \delta^{k+1}u^{k+1}, \Delta_k u^k, a, f) = 0, \quad \forall k, \forall \sigma \in K_k, \quad (\text{mPDE})$$

where u^k are unknown cochains, a encodes material parameters at their natural degree, f a source, and $\mathcal{F}_k$ a possibly nonlinear local functional. The structure of (mPDE) is what makes the problem hard for existing architectures: the equation at degree k couples u^k to cochains one degree below via $d^{k-1}u^{k-1}$ and one degree above via $\delta^{k+1}u^{k+1}$ — cross-dimensional coupling that neither a grid nor a graph can express. Maxwell, incompressible Navier–Stokes, and linear elasticity are all instances of (mPDE); single-degree equations are the degenerate exception.

A.3.1 Example PDEs

We illustrate (Evol) with two standard systems whose fields live on different cochain degrees and couple through d and δ .

Example 1 (Discrete Maxwell-type coupling). Maxwell’s equations describe how electric and magnetic fields evolve together: a changing magnetic field induces an electric field, and a changing electric field, together with current, induces a magnetic field. On an oriented 3-complex K , the electric field is naturally represented as an edge cochain $E \in C^1$, while the magnetic flux is represented as a face cochain $B \in C^2$. The core discrete coupling has the form

$$\partial_t B = -d^1 E, \quad \partial_t E = \delta^2 B - J,$$

with constraints

$$d^2 B = 0, \quad \delta^1 E = \rho.$$

Thus E drives B through d^1 , which measures circulation around faces, while B drives E through δ^2 , which aggregates adjacent face fluxes onto edges. The essential point is not the specific physical normalization, but the cross-degree structure: fields living on different ranks of the complex interact through the exterior derivative and its adjoint.

Example 2 (Discrete wave-type coupling). The wave equation describes how a disturbance propagates through an elastic medium: a displacement at one point pulls on its neighbors, which pull on theirs, and the wave travels outward. Let $u \in C^0$ be a scalar displacement on vertices, and let $p \in C^1$ be an edge variable encoding local stretch or momentum-like flux. A first-order discrete wave system can be written schematically as

$$\partial_t u = \delta^1 p, \quad \partial_t p = -c^2 d^0 u,$$

where c is the wave speed. Here d^0 sends vertex displacements to edge differences, while δ^1 aggregates edge quantities back to vertices. Eliminating p gives

$$\partial_t^2 u = -c^2 \delta^1 d^0 u = -c^2 \Delta_0 u,$$

showing that the wave operator itself is built from the same cross-degree maps.

B PDE Templates as Typed Cochain Systems

The previous sections define the main objects of the framework: a physical state is a tuple of cochains, the operators d, δ, Δ provide the allowed transport between and within degrees, and a TNO layer learns how to mix the transported features without learning the incidence support itself. The purpose of this section is to make this correspondence explicit for standard PDE systems.

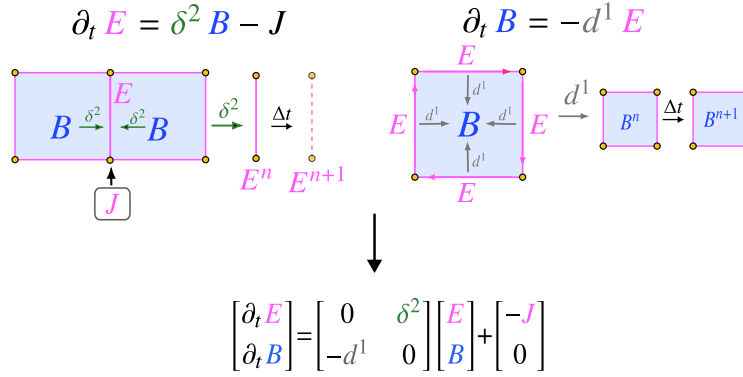


Figure 12: Maxwell coupling as cross-rank cochain transport. The electric field E lives on edges and the magnetic flux B lives on faces. Faraday’s law uses d^1 to map edge circulation to face updates, while the Ampère–Maxwell law uses δ^2 to map neighboring face fluxes back to edge updates, with source J . Together, these two typed updates form the block Maxwell system.

A useful example to explain this correspondence is Maxwell’s equations illustrated in Figure 12. In a compatible discretization, the electric field is not naturally a scalar value at vertices; it is a circulation along oriented edges. Likewise, the magnetic field is a flux through oriented faces. Thus

$$E \in C^1(K), \quad B \in C^2(K).$$

The discrete curl is the coboundary

$$d^1 : C^1(K) \rightarrow C^2(K),$$

which sends edge circulations to face fluxes. Its metric adjoint,

$$\delta^2 : C^2(K) \rightarrow C^1(K),$$

sends face fluxes back to edge-supported quantities. Ignoring material weights for notational clarity, the semi-discrete Maxwell system has the typed form

$$\partial_t B = -d^1 E, \quad \partial_t E = \delta^2 B - J,$$

or, equivalently,

$$\partial_t \begin{bmatrix} E \\ B \end{bmatrix} = \begin{bmatrix} 0 & \delta^2 \\ -d^1 & 0 \end{bmatrix} \begin{bmatrix} E \\ B \end{bmatrix} - \begin{bmatrix} J \\ 0 \end{bmatrix}.$$

This is precisely the kind of coupling encoded by the TNO block: the equation does not merely say that two feature arrays interact; it specifies that one field lives on edges, the other on faces, and that

Table 5: PDE templates as typed cochain systems. The DEC column shows the structural skeleton of the equation, while the TNO column shows the corresponding learned nonlinear cochain map.

PDE / system	DEC core structure	TNO generalization	Practical challenge / insight
Poisson / diffusion	$\Delta_0 u = f, \quad \Delta_0 = \delta^1 d^0$	$H_{\text{out}}^0 = \phi_\theta(H^0, \Delta_0 H^0 W_\Delta, f)$	Learns a Green-type solution operator on irregular meshes.
Heat equation	$\partial_t u + \Delta_0 u = f$	$H_{t+\Delta t}^0 = \phi_\theta(H_t^0, \Delta_0 H_t^0 W_\Delta, f_t, t)$	Time enters as a feature, recurrent state, or learned time-step map.
Advection–diffusion reaction	$\partial_t c + \mathcal{L}_v c = \kappa \Delta_0 c + R(c)$	$H_{\text{out}}^0 = \phi_\theta(H^0, \mathcal{L}_v H^0, \Delta_0 H^0 W_\Delta, R_\theta(H^0))$	Needs stable advection, upwinding, and control of stiff reactions.
Darcy / mixed elliptic	$q = -a d^0 p,$ $\delta^1 q = f$	$H_{\text{out}}^{0:1} = \phi_\theta(\mathcal{B}_{01} H^{0:1} W, a, f)$	Pressure and flux are kept as different cochain types; conservation is native.
Conservation law	$\partial_t \rho + \delta^1 q = s$	$H_{\text{out}}^{0:1} = \phi_\theta(H^0, H^1, \delta^1 H^1, s)$	Telescoping conservation follows from incidence algebra.
Wave, first-order	$\partial_t z = \mathcal{A}_{01} z,$ $z = (u, p)$	$H_{t+\Delta t}^{0:1} = \phi_\theta(H_t^{0:1}, \mathcal{A}_{01} H_t^{0:1})$	Same cross-rank structure as Darcy, but used for evolution; energy matters.
Maxwell	$\partial_t z = \mathcal{A}_{12} z - (J, 0),$ $z = (E, B)$	$H_{\text{out}}^{1:2} = \phi_\theta(\mathcal{B}_{12} H^{1:2} W, J)$	Separates electric circulation from magnetic flux; gauge and Gauss constraints matter.
Incompressible Navier–Stokes	$\partial_t u + \mathcal{L}_u u = -\nabla p + \nu \Delta_1 u,$ $\delta^1 u = 0$	$H_{\text{out}}^1 = \Pi_{\delta=0} \phi_\theta(H^1, \mathcal{L}_{H^1} H^1, \Delta_1 H^1 W_\Delta, \delta^2 H^2 W_\delta)$	Nonlinear advection must be paired with a hard divergence-free constraint.
Compressible Euler / NS	Continuity, momentum, and energy equations with fluxes	$H_{\text{out}}^{0:2} = \phi_\theta(H^{0:2}, dH, \delta H, \mathcal{F}_\theta(H^{0:2}))$	Requires shock-aware fluxes, positivity, and entropy consistency.
Elasticity	$\text{div } \sigma = f,$ $\sigma = \mathbb{C} : \varepsilon(u),$ $\varepsilon(u) = \frac{1}{2}(d^0 u + (d^0 u)^\top)$	$H_{\text{out}}^1 = \phi_\theta(H^1, \text{sym}(d^0 H^0) W_\varepsilon, \Delta_1 H^1 W_\Delta, \mathbb{C})$	The symmetric gradient and constitutive law are the load-bearing structures.
Shallow water	$\partial_t h + \delta^1(hu) = 0,$ momentum balance	$H_{\text{out}}^{0:1} = \phi_\theta(H^0, H^1, \delta^1(H^0 H^1), d^0 H^0, b)$	Couples height, flux, bathymetry, wetting/drying, and positivity.
Vorticity–stream	$\partial_t \omega + \{\psi, \omega\} = \nu \Delta_2 \omega,$ $\Delta_2 \psi = \omega$	$H_{\text{out}}^2 = \phi_\theta(H^2, \text{Poisson}_\theta(H^2), \mathcal{L}_{H^1} H^2, \Delta_2 H^2 W_\Delta)$	The Poisson / Biot–Savart solve is the global component.
Mean-field games Optimal transport	Continuity coupled with Hamilton–Jacobi–Bellman	$(\rho_{\text{out}}, \varphi_{\text{out}}) = \phi_\theta(\rho, \varphi, d^0 \varphi, \Delta_0 \varphi, \delta^1(\rho d^0 \varphi))$	Requires monotonicity, Wasserstein geometry, and stable density–potential coupling.
General k -form Hodge flow	$\partial_t u^k + \Delta_k u^k = f^k,$ $\Delta_k = \Delta_k^\uparrow + \Delta_k^\downarrow$	$H_{\text{out}}^k = \phi_\theta(H^k, \Delta_k^\uparrow H^k W_\uparrow, \Delta_k^\downarrow H^k W_\downarrow, f^k)$	Universal form-valued PDE template; only the active operator block changes.

$$\mathcal{B}_{01} = \begin{bmatrix} \Delta_0 & \delta^1 \\ d^0 & \Delta_1^\downarrow \end{bmatrix}, \quad \mathcal{B}_{12} = \begin{bmatrix} \Delta_1^\uparrow & \delta^2 \\ d^1 & \Delta_2^\downarrow \end{bmatrix}, \quad \mathcal{A}_{01} = \begin{bmatrix} 0 & \delta^1 \\ d^0 & 0 \end{bmatrix}, \quad \mathcal{A}_{12} = \begin{bmatrix} 0 & \delta^2 \\ -d^1 & 0 \end{bmatrix}.$$

the interaction must pass through the incidence operators of the complex. The corresponding learned cochain update has the same structural support,

$$\begin{bmatrix} H_{\text{out}}^1 \\ H_{\text{out}}^2 \end{bmatrix} = \phi_\theta \left(\begin{bmatrix} \Delta_1^\uparrow & \delta^2 \\ d^1 & \Delta_2^\downarrow \end{bmatrix} \begin{bmatrix} H^1 \\ H^2 \end{bmatrix} W, J \right).$$

$$\begin{aligned}
\begin{bmatrix} \partial_t E \\ \partial_t B \end{bmatrix} &= \begin{bmatrix} 0 & \delta^2 \\ -d^1 & 0 \end{bmatrix} \begin{bmatrix} E \\ B \end{bmatrix} + \begin{bmatrix} -J \\ 0 \end{bmatrix} & \begin{bmatrix} H_{\text{out}}^1 \\ H_{\text{out}}^2 \end{bmatrix} &= \phi_\theta \left(\begin{bmatrix} \Delta_1^\uparrow & \delta^2 \\ d^1 & \Delta_2^\downarrow \end{bmatrix} \begin{bmatrix} H^1 \\ H^2 \end{bmatrix} W, J \right) \\
\text{(a) Typed Maxwell coupling.} & & \text{(b) Induced TNO layer.}
\end{aligned}$$

Figure 13: From Maxwell coupling to a TNO template. **(a)** The Maxwell system is a typed cochain system: the electric field E lives on edges, the magnetic flux B lives on faces, and the PDE couples them through the cross-rank DEC maps $d^1 : C^1 \rightarrow C^2$ and $\delta^2 : C^2 \rightarrow C^1$, with source J acting on the edge equation. **(b)** The corresponding TNO layer keeps this typed routing. The off-diagonal blocks d^1 and δ^2 transport information between edge and face cochains, while the diagonal Hodge-Laplacian channels $\Delta_1^\uparrow$ and $\Delta_2^\downarrow$ perform same-rank propagation. Channel maps act on the feature dimension after DEC transport and are suppressed in the visual notation for readability. Learning enters through the channel weights $W^\bullet$ and the nonlinear update ϕ_θ , which determine how the routed features are mixed rather than where information can flow.

Here H^1 and H^2 are hidden edge and face cochains. The block matrix fixes where information can flow: edge-to-face through d^1 , face-to-edge through δ^2 , and same-rank propagation through the Hodge Laplacian channels. The source $J \in C^1(K)$ is an external edge-supported input to the update, not part of the DEC transport block. The trainable part, W and ϕ_θ , learns how these transported features are combined. The signs and physical constants in the Maxwell operator may be fixed explicitly or absorbed into the learned channel maps; the displayed TNO block is meant to preserve the typed incidence support of the PDE. See Figure 13.

This example also shows the exact-sequence structure. Namely, the identity $d^2 d^1 = 0$ implies

$$\partial_t(d^2 B) = -d^2 d^1 E = 0.$$

Hence the magnetic Gauss constraint is preserved by the algebra of the complex. Similarly, $\delta^1 \delta^2 = 0$, together with the discrete charge-continuity law, gives the corresponding compatibility condition for the electric equation. These identities are inherited from the incidence structure, not learned as soft penalties.

Darcy (what we specifically evaluated). The Darcy / mixed-elliptic system is the case actually studied in Sec. 5. It exercises typed routing across all three cochain ranks of a 2D complex. The pressure is defined as a scalar at vertices, the Darcy flux is a line integral along oriented edges, and the diffusivity is face-supported (a scalar on faces in the isotropic case, a rank-2 tensor on faces in Sec. 5.3):

$$p \in C^0(K), \quad q \in C^1(K), \quad a \in C^2(K),$$

with constitutive and balance equations

$$q = -a d^0 p, \quad \delta^1 q = f.$$

A TNO block on this system carries hidden pressure and flux cochains H^0, H^1 and routes them through the $\mathcal{B}_{01}$ block of Table 5; the material field a enters at its native rank as a face-valued input channel that conditions the constitutive map. This is the synthetic mesh family we examine in Sec. 5.3: when the per-face orientation is constructed so that vertex projection is lossy by design, an architecture that ingests a only at C^0 cannot recover the operator, while the rank-2 ingestion path retains the information that makes the typed routing load-bearing. The same incidence algebra makes the residual $\delta^1 q - f$ a vertex-supported quantity computed exactly by the complex. Thus mass balance can be enforced, penalized, or projected at the correct cochain degree rather than approximated through an untyped graph message.

Together, Maxwell and Darcy cover two qualitatively distinct roles a typed coupling can play: evolutionary cross-rank dynamics with gauge structure (Maxwell), and an elliptic constraint with native discrete conservation (Darcy). Poisson activates a single 0-cochain and the scalar Hodge Laplacian Δ_0 . Fluids, elasticity, conservation laws, and transport systems introduce problem-specific nonlinearities, such as advection, constitutive maps, learned fluxes, projections, or positivity constraints. In all cases, the same principle remains: the PDE determines the typed transport, while the TNO learns the nonlinear cochain map supported on that transport.

Table 5 summarizes this correspondence and should be read as an unpacking of the multi-degree PDE form and the block TNO layer, rather than as a collection of separate architectures.

C Proofs

C.1 Proof of Prop. 4.2 [FNO-Recovery]

Proof. On a uniform periodic cubical complex, the rank-0 Hodge Laplacian $\Delta_{0,N} = \delta^1 d^0$ is translation invariant and is diagonalized by the discrete Fourier basis:

$$\Delta_{0,N} = \mathcal{F}_N^{-1} \Lambda_N \mathcal{F}_N.$$

The zero eigenspace is the space of constant 0-cochains, hence

$$P_0^{\text{harm}} u = \mathcal{F}_N^{-1}(\mathbf{1}_{\xi=0} \widehat{u}(\xi)), \quad (I - P_0^{\text{harm}})u = \mathcal{F}_N^{-1}(\mathbf{1}_{\xi \neq 0} \widehat{u}(\xi)).$$

Define the rank-0 spectral TNO channel

$$S_{\theta,N} u = \mathcal{F}_N^{-1}(R_{\theta}(\xi) \widehat{u}(\xi))_{\xi}$$

with the usual real-valuedness condition

$$R_{\theta}(-\xi) = \overline{R_{\theta}(\xi)}.$$

Equivalently, separating the harmonic and non-harmonic parts,

$$S_{\theta,N} u = \mathcal{F}_N^{-1}(\mathbf{1}_{\xi \neq 0} R_{\theta}(\xi) \widehat{u}(\xi)) + \mathcal{F}_N^{-1}(\mathbf{1}_{\xi=0} R_{\theta}(0) \widehat{u}(0)).$$

Thus

$$S_{\theta,N} u = S_{\theta,N}(I - P_0^{\text{harm}})u + R_{\theta}(0)P_0^{\text{harm}}u.$$

Now choose the rank-0 TNO layer

$$T_{\theta,N}(u) = W_{\text{self}}u + S_{\theta,N}(I - P_0^{\text{harm}})u + W_{\text{harm}}P_0^{\text{harm}}u,$$

with

$$W_{\text{self}} = W, \quad W_{\text{harm}} = R_{\theta}(0).$$

Then

$$\begin{aligned} T_{\theta,N}(u) &= Wu + \mathcal{F}_N^{-1}(\mathbf{1}_{\xi \neq 0} R_{\theta}(\xi) \widehat{u}(\xi)) + \mathcal{F}_N^{-1}(\mathbf{1}_{\xi=0} R_{\theta}(0) \widehat{u}(0)) \\ &= Wu + \mathcal{F}_N^{-1} R_{\theta} \mathcal{F}_N u. \end{aligned}$$

This is exactly the linear FNO layer. $\square$

C.2 Proof of Prop. 4.3 [GNO-Recovery]

Proof. A graph neural operator approximates an integral operator of the form

$$(\mathcal{K}_{\theta}u)(x) = \int_D \kappa_{\theta}(x, y)u(y) d\mu(y),$$

where $\kappa_{\theta}(x, y)$ is a learned kernel acting on the feature value at the source point y and producing a contribution at the query point x . On a finite point cloud $\{x_i\}_{i=1}^n$, this integral is replaced by a quadrature rule over a neighborhood $\mathcal{N}(i)$ of each query point:

$$(\mathcal{K}_{\theta}u)(x_i) \approx \sum_{j \in \mathcal{N}(i)} \kappa_{\theta}(x_i, x_j)u_j \omega_j,$$

where $\omega_j > 0$ is the quadrature weight associated with the source point x_j . A GNO layer then adds a pointwise channel map:

$$u_i \mapsto Wu_i + \sum_{j \in \mathcal{N}(i)} \kappa_{\theta}(x_i, x_j)u_j \omega_j.$$

We now realize this update as a rank-0 graph-quadrature TNO. Construct a directed graph complex $G = (V, E)$ with one vertex v_i for each sample point x_i . For every quadrature interaction $j \in \mathcal{N}(i)$, add a directed rank-1 cell

$$e_{ij} : v_j \rightarrow v_i.$$

Thus each edge represents one source-to-target quadrature contribution.

Define a rank-1 cochain $h^1 \in C^1(G; \mathbb{R}^{d_h})$ by assigning to each directed edge the corresponding learned kernel message:

$$h^1(e_{ij}) = \kappa_\theta(x_i, x_j) u_j \omega_j.$$

This is a cellular message: it lives on the rank-1 cell e_{ij} and is supported only where the graph complex contains an incidence from source v_j to target v_i .

Let

$$S_{\text{tar}} : C^1(G; \mathbb{R}^{d_h}) \rightarrow C^0(G; \mathbb{R}^{d_h})$$

denote the target-incidence aggregation operator

$$(S_{\text{tar}} h^1)_i = \sum_{e_{ij} : \text{tar}(e_{ij}) = v_i} h^1(e_{ij}) = \sum_{j \in \mathcal{N}(i)} h^1(e_{ij}).$$

Equivalently, S_{tar} is the unsigned incoming-incidence matrix of the directed graph. It is important that this is the incoming-incidence aggregation, not the signed boundary operator: the signed boundary would also include source signs and would not equal the GNO quadrature sum in general.

Substituting the definition of h^1 gives

$$(S_{\text{tar}} h^1)_i = \sum_{j \in \mathcal{N}(i)} \kappa_\theta(x_i, x_j) u_j \omega_j.$$

Therefore the rank-0 TNO update with self-channel $W_0^{\text{self}} = W$ is

$$\mathcal{T}_{\theta,0}(u)_i = W u_i + (S_{\text{tar}} h^1)_i = W u_i + \sum_{j \in \mathcal{N}(i)} \kappa_\theta(x_i, x_j) u_j \omega_j.$$

This is exactly the GNO graph-quadrature update. Applying the pointwise nonlinearity after this affine update gives the usual nonlinear GNO layer. $\square$

C.3 Cell-wise Realization and Copsheaf Variant

The block update in Sec. 4.2 can be written cell by cell. This form makes explicit that a TNO layer is local, but with locality defined by DEC rather than by an arbitrary graph adjacency. The relevant neighborhoods are induced by the routes d , δ , and the two Hodge–Laplacian channels $\Delta^\uparrow, \Delta^\downarrow$.

Let $\tilde{K}$ be the lifted complex used by the network, and let $\tilde{K}_k$ denote its k -cells. For a hidden k -cochain $h^k \in C^k(\tilde{K}; \mathbb{R}^{d_k})$, write

$$h_\sigma^k := h^k(\sigma), \quad \sigma \in \tilde{K}_k.$$

For a target k -cell σ , define the four DEC neighborhoods

$$\mathcal{N}_{\text{ex}}^k(\sigma) = \{\rho \in \tilde{K}_{k-1} : \rho \prec \sigma\}, \quad \mathcal{N}_{\text{coex}}^k(\sigma) = \{\eta \in \tilde{K}_{k+1} : \sigma \prec \eta\},$$

and

$$\mathcal{N}_{\Delta^\uparrow}^k(\sigma) = \{\tau \in \tilde{K}_k : (\Delta_k^\uparrow)_{\sigma\tau} \neq 0\}, \quad \mathcal{N}_{\Delta^\downarrow}^k(\sigma) = \{\tau \in \tilde{K}_k : (\Delta_k^\downarrow)_{\sigma\tau} \neq 0\}.$$

The first two neighborhoods are cross-rank. The exact route $\mathcal{N}_{\text{ex}}^k$ receives from $(k-1)$ -cells through d^{k-1} , while the coexact route $\mathcal{N}_{\text{coex}}^k$ receives from $(k+1)$ -cells through δ^{k+1} . The last two neighborhoods are same-rank. The upper Laplacian route couples k -cells through common $(k+1)$ -cells, while the lower Laplacian route couples k -cells through common $(k-1)$ -cells. For $k=1$, these are edge-to-edge couplings through shared faces and shared vertices, respectively.

We now attach a copsheaf to each route. For each route

$$r \in \{\text{ex}, \text{coex}, \Delta^\uparrow, \Delta^\downarrow\},$$

the neighborhood function $\mathcal{N}_r^k$ induces a directed graph whose edges are

$$y \rightarrow \sigma \quad \text{whenever} \quad y \in \mathcal{N}_r^k(\sigma).$$

A route-dependent copsheaf assigns a feature space F_x to every cell x appearing in this directed graph and a linear transport map

$$\rho_{y \rightarrow \sigma}^r : F_y \rightarrow F_\sigma$$

to every directed neighborhood edge $y \rightarrow \sigma$. In implementation, one often takes $F_\sigma \simeq \mathbb{R}^{d_k}$ for all k -cells σ , but the notation allows different ranks, cell types, or incidences to carry different feature spaces.

The DEC scalar coefficient along each route is kept separate from the copresheaf map. Define

$$a_{\sigma\rho}^{k,\text{ex}} = [\sigma : \rho], \quad a_{\sigma\eta}^{k,\text{coex}} = (\delta^{k+1})_{\sigma\eta},$$

and

$$a_{\sigma\tau}^{k,\Delta^\uparrow} = (\Delta_k^\uparrow)_{\sigma\tau}, \quad a_{\sigma\tau}^{k,\Delta^\downarrow} = (\Delta_k^\downarrow)_{\sigma\tau}.$$

The coefficient $a^{k,\text{ex}}$ is purely incidence and orientation. The coefficients involving δ and Δ carry the metric or material weights induced by the Hodge stars. The learned copresheaf map $\rho_{y \rightarrow \sigma}^r$ then transports the source feature into the target feature space. Equivalently, the scalar DEC coefficient could be absorbed into $\rho_{y \rightarrow \sigma}^r$, but separating them makes clear what is fixed by DEC and what is learned.

The cell-wise update has four routed copresheaf channels:

$$\begin{aligned} m^{k,\text{ex}}(\sigma) &= \bigoplus_{\rho \in \mathcal{N}_{\text{ex}}^k(\sigma)} \psi_\theta^{\text{ex}}(h_\sigma^k, a_{\sigma\rho}^{k,\text{ex}} \rho_{\rho \rightarrow \sigma}^{\text{ex}} h_\rho^{k-1}). \\ m^{k,\text{coex}}(\sigma) &= \bigoplus_{\eta \in \mathcal{N}_{\text{coex}}^k(\sigma)} \psi_\theta^{\text{coex}}(h_\sigma^k, a_{\sigma\eta}^{k,\text{coex}} \rho_{\eta \rightarrow \sigma}^{\text{coex}} h_\eta^{k+1}). \\ m^{k,\Delta^\uparrow}(\sigma) &= \bigoplus_{\tau \in \mathcal{N}_{\Delta^\uparrow}^k(\sigma)} \psi_\theta^{\Delta^\uparrow}(h_\sigma^k, a_{\sigma\tau}^{k,\Delta^\uparrow} \rho_{\tau \rightarrow \sigma}^{\Delta^\uparrow} h_\tau^k). \\ m^{k,\Delta^\downarrow}(\sigma) &= \bigoplus_{\tau \in \mathcal{N}_{\Delta^\downarrow}^k(\sigma)} \psi_\theta^{\Delta^\downarrow}(h_\sigma^k, a_{\sigma\tau}^{k,\Delta^\downarrow} \rho_{\tau \rightarrow \sigma}^{\Delta^\downarrow} h_\tau^k). \end{aligned}$$

Here $\bigoplus$ is a permutation-invariant aggregation, such as a sum, mean, or attention-weighted sum. Each message first transports the source feature through the copresheaf map $\rho_{y \rightarrow \sigma}^r$, then weights it by the DEC coefficient of the corresponding route, and finally applies the learnable message function ψ_θ^r .

The output feature at σ is

$$h_{\text{out}}^k(\sigma) = \phi_\theta(h_\sigma^k, m^{k,\text{ex}}(\sigma), m^{k,\text{coex}}(\sigma), m^{k,\Delta^\uparrow}(\sigma), m^{k,\Delta^\downarrow}(\sigma)).$$

The functions ψ_θ^r form route-wise messages, while ϕ_θ mixes the self feature with the four routed channels. The DEC operators determine which cells can communicate and provide the orientation/Hodge coefficients. The copresheaf maps determine how features are transported between the local feature spaces attached to the source and target cells.

In operator form, the route-wise copresheaf transports are

$$\begin{aligned} (d_\rho^{k-1} h^{k-1})_\sigma &= \sum_{\rho \prec \sigma} [\sigma : \rho] \rho_{\rho \rightarrow \sigma}^{\text{ex}} h_\rho^{k-1}, \\ (\delta_\rho^{k+1} h^{k+1})_\sigma &= \sum_{\eta \succ \sigma} (\delta^{k+1})_{\sigma\eta} \rho_{\eta \rightarrow \sigma}^{\text{coex}} h_\eta^{k+1}, \end{aligned}$$

and

$$\begin{aligned} (\Delta_{k,\rho}^\uparrow h^k)_\sigma &= \sum_{\tau \in \mathcal{N}_{\Delta^\uparrow}^k(\sigma)} (\Delta_k^\uparrow)_{\sigma\tau} \rho_{\tau \rightarrow \sigma}^{\Delta^\uparrow} h_\tau^k, \\ (\Delta_{k,\rho}^\downarrow h^k)_\sigma &= \sum_{\tau \in \mathcal{N}_{\Delta^\downarrow}^k(\sigma)} (\Delta_k^\downarrow)_{\sigma\tau} \rho_{\tau \rightarrow \sigma}^{\Delta^\downarrow} h_\tau^k. \end{aligned}$$

Thus the copresheaf variant replaces the fixed DEC transports by route-wise transported DEC operators:

$$d^{k-1} \rightsquigarrow d_\rho^{k-1}, \quad \delta^{k+1} \rightsquigarrow \delta_\rho^{k+1}, \quad \Delta_k^\uparrow, \Delta_k^\downarrow \rightsquigarrow \Delta_{k,\rho}^\uparrow, \Delta_{k,\rho}^\downarrow.$$

The fixed DEC layer is recovered by taking the copresheaf maps to be identity maps, or shared channel maps independent of the incident pair. Then the only route coefficients are the signed incidence coefficients and the Hodge-weighted DEC coefficients. The copresheaf version keeps the same DEC supports but allows the feature transport itself to vary by route, rank, orientation, geometry, cell type, or learned attributes. This gives a controlled extension of the DEC layer: the topology fixes the admissible routes, while the copresheaf maps learn how features are translated along those routes.

D TNOs, Hodge-Compatible Smoothing, and the Multigrid Correspondence

We now explain the numerical-analysis interpretation of TNO and HTNO. The point is not that a trained TNO inherits the convergence theory of classical multigrid. It does not. Rather, the claim is structural: the four channels of the TNO layer, the use of residual depth, and the hierarchical construction of HTNO reproduce the algebraic ingredients that make multigrid work for de Rham complexes. These are precisely the ingredients missing from ordinary point smoothers on vector-valued nodal fields.

D.1 Why scalar multigrid does not directly extend

How scalar multigrid works. For the scalar Poisson problem

$$-\Delta_0 u = f, \quad u \in C^0(K),$$

geometric multigrid succeeds because of a clean separation of scales in the error. The error $e = u - u_h$ between the true solution and the current iterate can be thought of as a sum of components that oscillate at different spatial frequencies on the mesh. After a few sweeps of a local smoother—a simple iterative method such as Gauss–Seidel or damped Jacobi that updates each unknown from a small patch of the mesh—the high-frequency (rapidly oscillating) part of the error is reduced efficiently. What remains varies slowly across the mesh and is therefore well represented on a coarser version of the same grid. The coarse-grid correction step solves a cheaper problem there and returns a correction to the fine grid; cycling between levels yields a combined iteration whose convergence rate is independent of the mesh size under standard ellipticity and approximation assumptions [82, 21].

The critical point is that for $-\Delta_0$ every error component is handled by one of the two mechanisms: either the smoother kills it, or the coarse grid represents it. No part of the error escapes both.

The curl–curl problem and its large kernel. This clean picture breaks for problems involving differential forms, that is, for unknowns that live on edges, faces, or cells of the mesh rather than on nodes. Consider the curl–curl block

$$\delta^2 d^1 e + \tau e = f, \quad e \in C^1(K),$$

where e is an edge-valued unknown (a discrete 1-form), d^1 is the discrete curl operator (mapping edges to faces), and δ^2 is its L^2 -adjoint (mapping faces back to edges). This equation appears in time-harmonic Maxwell equations, magnetostatic vector-potential formulations, and the degree-one Hodge Laplacian—the natural generalization of the scalar Laplacian to edge-valued fields [62, 63, 18, 42, 5, 6]. When $\tau = 0$ the operator $\delta^2 d^1$ is only positive semidefinite; even for small $\tau > 0$ the near-null space is so large that the same practical difficulties arise.

The source of trouble is a fundamental identity in the de Rham complex. The discrete gradient d^0 maps node values to edge values (it assigns to each edge the signed difference of the node values at its endpoints), and the discrete curl d^1 maps edge values to face values. These two operators satisfy

$$d^1 d^0 = 0,$$

which is simply the fact that the curl of any gradient is zero. Consequently, every edge cochain of the form $d^0 \phi$ —a discrete gradient field—is in the kernel of the curl–curl operator:

$$\delta^2 d^1 d^0 \phi = 0.$$

This means

$$\text{im } d^0 \subset \ker(\delta^2 d^1),$$

and the dimension of this invisible subspace is

$$\dim \text{im } d^0 = \dim C^0 - \beta_0,$$

where β_0 is the number of connected components of the domain (typically one). On topologically nontrivial domains—those with loops or handles—there are further null modes corresponding to harmonic representatives, fields that are curl-free and divergence-free but not globally a gradient [42, 5, 6].

Why a smoother is blind to gradient error. To see concretely why this causes trouble, suppose the current error e contains a gradient component $e_{\text{grad}} = d^0 \phi$ for some node cochain ϕ . This component contributes nothing to the curl–curl residual:

$$\delta^2 d^1 e_{\text{grad}} = 0.$$

A smoother that updates the edge unknowns using only the curl–curl residual—regardless of how many sweeps are applied—never sees this component and therefore never reduces it. The iteration is not merely *slow* on gradient error; it is *completely blind* to it. This is a qualitative failure, not a quantitative one.

Compare with the scalar Poisson case: there, the Laplacian $-\Delta_0$ has a null space of dimension one (constant functions on a closed domain), handled by fixing a single degree of freedom. Here the null space has dimension equal to the number of nodes, comparable to the total number of unknowns. It cannot be removed by a simple constraint; it must be explicitly visited by the solver at every level.

The Hodge decomposition and three-component obstruction. The full picture is given by the Hodge decomposition, which states that any edge cochain can be written as the sum of three mutually orthogonal pieces [4–6]:

$$C^k = \underbrace{\text{im } d^{k-1}}_{\text{exact (gradient)}} \oplus \underbrace{\text{im } \delta^{k+1}}_{\text{coexact}} \oplus \underbrace{\mathcal{H}^k}_{\text{harmonic}},$$

where $\mathcal{H}^k = \ker d^k \cap \ker \delta^k$ is the space of fields that are simultaneously curl-free and divergence-free. Each piece behaves differently under the Hodge Laplacian $\Delta_k = \delta^{k+1} d^k + d^{k-1} \delta^k$:

- The **exact part** $\text{im } d^{k-1}$ lies in the kernel of the upper block $\delta^{k+1} d^k$; it is governed entirely by the lower block $d^{k-1} \delta^k$ and is invisible to any smoother that acts only through the upper block.
- The **coexact part** $\text{im } \delta^{k+1}$ lies in the kernel of the lower block; it is governed by the upper block and is what a standard curl–curl smoother can address.
- The **harmonic part** $\mathcal{H}^k$ lies in the kernel of both blocks and is in the null space of Δ_k ; its dimension equals the k -th Betti number β_k , a topological invariant of the domain.

A scalar smoother acting through a single energy can handle at most one of these three components. The other two are left entirely untouched. This is the fundamental obstruction: the difficulty is not that the unknown is edge-valued, but that the operator has a topologically structured null and near-null space that a uniform local relaxation cannot see [42, 4–6].

Consequences for solver design. Robust solvers for $H(\text{curl})$, $H(\text{div})$, and Hodge–Laplacian problems must therefore be designed to address all three Hodge components explicitly:

- **Coarse spaces** must carry a compatible discrete complex—a coarse-level version of the gradient, curl, and divergence operators that mirrors the fine-level structure, so that gradient error on the fine grid is represented as gradient error on the coarse grid.
- **Transfer operators** (prolongation and restriction) must commute with the discrete differential operators d^k , so that a gradient on the coarse grid prolongs to a gradient on the fine grid—not to a spurious mix of components.
- **Smoothers** must address all three Hodge components, either by augmenting the edge smoother with a nodal (scalar) solve that targets gradient error—Hiptmair’s subspace decomposition smoother [41]—or by constructing block smoothers that simultaneously see both blocks of Δ_k .

These requirements are the mechanism behind specialized Maxwell multigrid, auxiliary-space preconditioners [43], and FEEC-based multilevel methods [41, 4, 42, 5, 6, 3].

D.2 Hiptmair smoothing

Hiptmair’s remedy is to smooth not only on the space of the unknown, but also on the neighboring spaces in the de Rham complex [41]. For an edge variable $e \in C^1(K)$, the smoother combines:

1. a relaxation on C^1 , which acts on the coexact component;
2. an auxiliary relaxation on C^0 , lifted by d^0 , which acts on the exact component [41, 43];
3. a harmonic correction when $\beta_1 > 0$ [41].

This matches the discrete Hodge decomposition [44, 78, 5, 6]

$$C^1(K) = \text{im } d^0 \oplus \mathcal{H}^1 \oplus \text{im } \delta^2.$$

At degree k , the same principle uses the neighboring spaces C^{k-1} and C^{k+1} together with the harmonic subspace $\mathcal{H}^k = \ker \Delta_k$ [41, 4, 5]. The lesson is simple but important: a Hodge-compatible smoother for k -cochains must communicate with adjacent cochain degrees through the discrete differential and codifferential. This is not an implementation detail; it is what removes the kernel blindness of ordinary local relaxation [41, 42, 4].

D.3 The TNO layer as a Hodge-structured update

The linear TNO layer at degree k has the form

$$T_{\theta,k}(h) = d^{k-1}h^{k-1}W_k^\downarrow + \delta^{k+1}h^{k+1}W_k^\uparrow + P_k^{\text{harm}}h^k W_k^{\text{harm}} + h^k W_k^{\text{self}}.$$

The first three terms land in the three Hodge components of C^k [44, 5, 6]:

$$\text{im } d^{k-1}, \quad \text{im } \delta^{k+1}, \quad \mathcal{H}^k = \ker \Delta_k.$$

The final term is a local residual channel on the full space C^k . The branch $d^{k-1}h^{k-1}$ moves lower-degree information into degree k through the exact component. This is analogous to the auxiliary-space correction in Hiptmair-type smoothers, where a lower-degree potential is lifted into the target cochain degree [41, 43]. The branch $\delta^{k+1}h^{k+1}$ moves higher-degree information into degree k through the coexact component. The harmonic branch $P_k^{\text{harm}}h^k$ extracts the component not seen by either d^k or δ^k , while the self-channel provides a local update on C^k . Thus the layer has the same typed correction paths used by Hodge-decomposition-based smoothers [41, 4, 5], but with learned channel maps in place of fixed scalar relaxation parameters. This is a structural analogy rather than a multigrid convergence statement: a classical smoother updates solver residuals, whereas a TNO layer updates learned cochain features.

D.4 Depth as preconditioned iteration

The residual TNO update

$$h_{\ell+1}^k = h_\ell^k + H_\theta^k(h_\ell^{k-1}, h_\ell^k, h_\ell^{k+1}, d^{k-1}h_\ell^{k-1}, \delta^{k+1}h_\ell^{k+1}, \Delta_k h_\ell^k, a, f)$$

has the form of a stationary residual iteration [74]. If A is the discrete PDE operator and B_θ is the learned Hodge-compatible preconditioner encoded by the layer, then the idealized update is

$$h_{\ell+1} = h_\ell + B_\theta^{-1}(f - Ah_\ell),$$

where $B_\theta^{-1}(\cdot)$ denotes the action of the preconditioner. Stacking L residual layers is therefore analogous to applying L steps of a preconditioned Richardson iteration [74, 82, 22]. This gives a concrete interpretation of depth: it is not merely additional expressivity, it is additional correction time. The diminishing returns observed in our experiments with increasing depth are consistent with this solver interpretation: once the dominant residual components have been reduced, further iterations contribute less.

Residual layers and physical structure. When the target $\mathcal{G}$ arises from a discrete PDE, the architecture should reflect the structure of that law. A *residual TNO layer* therefore keeps the DEC operators d^k, δ^k, Δ_k as fixed transport scaffolding determined by the cell complex and its Hodge stars, and learns only how the transported cochain features are mixed and corrected. Writing $\mathbf{H}_\ell = (H_\ell^0, \dots, H_\ell^N)$ for the hidden cochains at layer ℓ on the lifted complex $\tilde{K}$, the residual form of $\mathbf{H}_{\text{out}}$ (as in Sec. 4.2) is

$$\mathbf{H}_{\ell+1} = \mathbf{H}_\ell + \phi_\theta(\mathcal{D} \mathbf{H}_\ell W), \quad (17)$$

where $\mathcal{D}$ is the block Dirac-type operator in Sec. 4.2 [13, 24], with Δ_k on the diagonal and d^{k-1}, δ^{k+1} on the off-diagonals. Coefficient and source cochains a, f enter as additional input channels at the appropriate ranks, consistent with (P1). Per rank, this expands to

$$H_{\ell+1}^k = H_\ell^k + \phi_\theta \left(H_\ell^k, d^{k-1} H_\ell^{k-1} W_k^\uparrow, \delta^{k+1} H_\ell^{k+1} W_k^\downarrow, \Delta_k^\uparrow H_\ell^k W_k^{\Delta,\uparrow}, \Delta_k^\downarrow H_\ell^k W_k^{\Delta,\downarrow} \right), \quad (18)$$

with out-of-range terms omitted. The residual connection stabilizes training and makes the identity easy to represent, which is useful for small- Δt time-stepping operators.

Per-layer pseudocode. Alg. 1 unrolls one residual TNO layer in the form actually computed at runtime: each rank- k block is updated from its own state and the transported contributions of its neighboring ranks, with the supports prescribed by the DEC operators of $\tilde{K}$. The block operator $\mathcal{D}$ of Sec. 4.2 already encodes the cross-rank routing, so no separate message-passing scaffolding is required: applying $\mathcal{D}$ to $\mathbf{H}_\ell$ is a single sparse block-matrix multiply that simultaneously realizes all three channels (exact, coexact, Laplacian) at every rank.

Input: Cochain features $\mathbf{H}_\ell = (H_\ell^0, \dots, H_\ell^N)$; DEC operators $\{d^{k-1}\}, \{\delta^{k+1}\}, \{\Delta_k^\uparrow\}, \{\Delta_k^\downarrow\}$ of $\tilde{K}$; channel-mixing matrices $\{W_k^\downarrow, W_k^\uparrow, W_k^{\Delta,\uparrow}, W_k^{\Delta,\downarrow}\}$; feature transformation ϕ_θ

Output: Updated features $\mathbf{H}_{\ell+1} = (H_{\ell+1}^0, \dots, H_{\ell+1}^N)$

for $k = 0, 1, \dots, N$ **do**

$$\begin{aligned} m_k^\downarrow &\leftarrow d^{k-1} H_\ell^{k-1} W_k^\downarrow; & // \text{ exact channel} \\ m_k^\uparrow &\leftarrow \delta^{k+1} H_\ell^{k+1} W_k^\uparrow; & // \text{ coexact channel} \\ m_k^\Delta &\leftarrow \Delta_k^\uparrow H_\ell^k W_k^{\Delta,\uparrow} + \Delta_k^\downarrow H_\ell^k W_k^{\Delta,\downarrow}; & // \text{ Hodge-Laplacian channel} \\ H_{\ell+1}^k &\leftarrow H_\ell^k + \phi_\theta(H_\ell^k, m_k^\downarrow, m_k^\uparrow, m_k^\Delta); & // \text{ cf. Eq. (9)} \end{aligned}$$

end

return $\mathbf{H}_{\ell+1}$

Algorithm 1: Residual TNO layer (per layer ℓ , on lifted complex $\tilde{K}$)

D.5 HTNO as a learned two-grid method

HTNO adds a coarse complex K^c and degree-wise transfer maps

$$R_k : C^k(K) \rightarrow C^k(K^c), \quad \Pi_k : C^k(K^c) \rightarrow C^k(K).$$

For de Rham systems, the ideal structure-preserving condition is that transfer commute with the coboundary:

$$d_{K^c}^k R_k = R_{k+1} d_K^k, \quad d_K^k \Pi_k = \Pi_{k+1} d_{K^c}^k.$$

This is the standard commuting-diagram condition in FEEC multigrid [4, 5, 3]. When these identities hold, taking a coboundary and transferring levels agrees with transferring first and then taking the coboundary. In particular, exact cochains remain exact across levels, and the discrete differential structure is preserved by restriction and prolongation. Without such compatibility, a coarse correction can turn a gradient-like error into a non-gradient error after prolongation, weakening the decomposition on which Hodge-type smoothers rely [41, 4].

In the present HTNO implementation, these commuting identities are not imposed as hard algebraic constraints. Rather, they serve as the FEEC ideal motivating the hierarchy. The TNO blocks on each level use the DEC operators of that level, so typed cochain transport is preserved level-wise. The inter-level maps used in our experiments are induced by a partition of the fine vertices and should therefore be viewed as cochain-compatible coarse correction maps, not as exact commuting-diagram transfers.

With this interpretation, an HTNO block is a learned two-grid V-cycle (Fig. 14) [82, 22] that pre-smooths on K with Hodge-structured TNO layers [41, 42, 4], restricts to K^c , computes an approximate correction on K^c with coarse-level TNO layers, prolongs the correction back to K , additively adds it to the pre-smoothed state, and post-smooths on K . With $L = 2$ this reads

$$\text{HTNO}_\theta(h) = \mathcal{T}_\theta^{\text{post}} \left(\mathcal{T}_\theta^{\text{pre}}(h) + \Pi_0^0 \mathcal{T}_\theta^c(R_0^0 \mathcal{T}_\theta^{\text{pre}}(h)) \right). \quad (19)$$

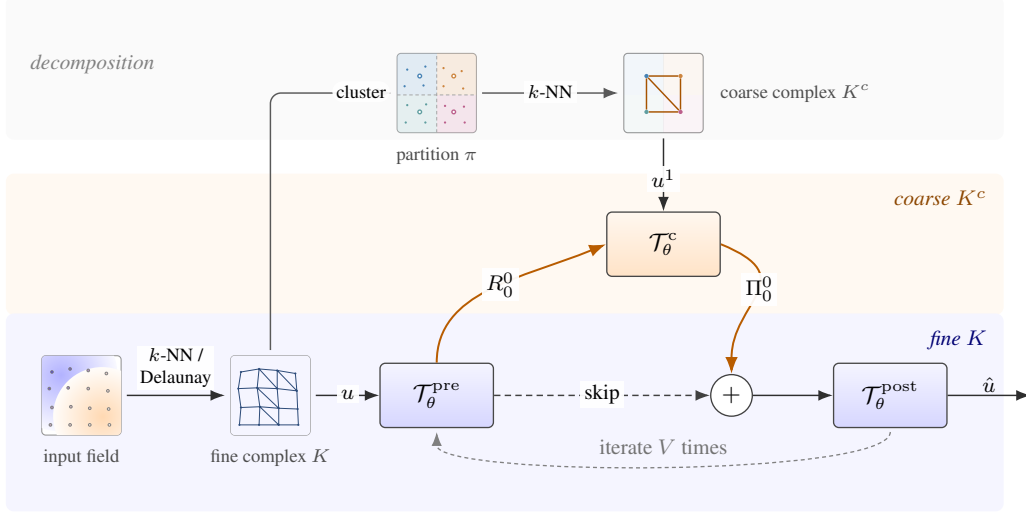


Figure 14: HTNO as a learned two-grid V-cycle. **Bottom:** the input field on a point cloud is lifted to a fine cell complex K (k -NN or Delaunay), giving an input cochain $u \in C^\bullet(K)$ that drives the V-cycle. The pre-smoother $\mathcal{T}_\theta^{\text{pre}}$ runs on K ; its output is restricted by R_0^0 to the coarse complex, where $\mathcal{T}_\theta^c$ computes a correction; the correction is prolonged back by Π_0^0 and added to the pre-smoothed state via the skip; $\mathcal{T}_\theta^{\text{post}}$ closes the cycle, producing $\hat{u}$. The whole block can be iterated V times with shared weights. **Top:** the coarse complex is built once from K . Fine vertices are clustered (fixed k -means or learned soft Voronoi) into a partition π , and k -NN on the resulting centroids yields the coarse complex K^c , on which the coarse cochain u^c lives.

Our implementation realizes R_0^0, Π_0^0 as mean-pool and broadcast over a partition $\pi : V(K) \rightarrow \{1, \dots, K_c\}$ of the fine vertices. The partition is either *fixed* (k -means on input coordinates, SLIC superpixels, or an MS-complex segmentation), or *learned end-to-end* as a soft Voronoi diagram: K_c trainable query vectors attend over the warm-up features to predict centroid positions, soft assignments are obtained from $W = \text{softmax}(-\|p - c\|^2/T)$ (or a Gaussian RBF kernel), and auxiliary spread and entropy losses on W keep the centroids from collapsing. The coarse complex K^c is built by k -NN over the centroids, inheriting its own incidence matrices and Hodge stars.

Higher-rank transfers ($k \geq 1$) would require structured prolongation and restriction maps, for example Whitney/Nédélec-style constructions, to satisfy the commuting identities exactly [65, 18, 5]. We do not impose such maps in the present implementation. Instead, we re-derive rank-1 and rank-2 features on K^c from the pooled vertex field using the coarse incidence structure. Thus $\mathcal{T}_\theta^c$ runs the multi-rank TNO update of Eq. (10) on K^c when higher-rank features are present, and a rank-0 message-passing block otherwise. The fine-coarse-fine block used in our experiments is the $V = 1$ instance of Eq. (19); iterating with shared weights for $V > 1$, or recursing on deeper hierarchies, gives learned multilevel V-cycles.

D.6 Scope of the analogy

The correspondence with Hodge-compatible smoothing is structural, not a multigrid convergence theorem. Classical multigrid convergence depends on a specified residual equation, smoother, transfer operators, and regularity assumptions [82, 22, 21]. A TNO layer does not inherit these rates automatically. What is shared is the algebraic form: cross-degree transport is fixed by d^{k-1} and δ^{k+1} , harmonic components are accessed through P_k^{harm} , and inter-level maps can be chosen to commute with d [4, 5]. Training then learns channel mixing, nonlinear correction, and coarse representations inside this fixed de Rham structure.

This distinction is also what separates the construction from point-only neural operators [48, 52]. A point-only model may use pooling, attention, U-Net skips, or learned coarse variables, but it does not by default carry typed cochain spaces C^k , fixed maps $d^k : C^k \rightarrow C^{k+1}$, codifferentials δ^{k+1} , or harmonic projectors. TNOs make these objects primitive operators of the architecture rather than behaviors to be inferred from data.

E Implementation Details

E.1 MPNN ablation baseline

The MPNN used in the ablations of Sec. 5.4 is a standard residual message-passing network on rank-0 (vertex) features only, sharing the Delaunay graph and the node/edge inputs of TNO but with no sheaf, harmonic, or higher-rank channels. Inputs $x_v \in \mathbb{R}^{F_{\text{in}}}$ and edge features $e_{ij} \in \mathbb{R}^{F_e}$ are lifted to width w by a $\text{Dense} \rightarrow \text{swish} \rightarrow \text{LayerNorm}$ encoder applied independently on vertices and edges. Each of the $L=12$ residual layers, with $\bar{h} = \text{LN}(h)$, computes

$$m_{ij} = \text{swish}(W_m [\bar{h}_i \parallel \bar{h}_j \parallel e_{ij}]), \quad a_i = \sum_{j: j \rightarrow i} m_{ij}, \quad (20)$$

$$h_i \leftarrow h_i + \text{LN}(\text{swish}(W_2 \text{swish}(W_1 [\bar{h}_i \parallel a_i]))), \quad (21)$$

with sum aggregation on the destination index. A final $\text{LN} \rightarrow \text{Dense}$ decoder produces the vertex output. The matched-width baseline uses $w=192$, the parameter-matched baseline uses $w=272$ (cf. Sec. F.5); all other optimizer/training settings are inherited from the ablation backbone.

E.2 Complexity of a TNO layer

Let $n_k = |\tilde{K}_k|$ be the number of rank- k cells, $n_\bullet = \sum_k n_k$, and $h = d_h$. Let

$$s_\bullet = \sum_{k=1}^N \text{nnz}(B_k), \quad \lambda_\bullet = \sum_{k=0}^N (\text{nnz}(\Delta_k^\uparrow) + \text{nnz}(\Delta_k^\downarrow)),$$

with "nnz" the non-zero counts for the sparse operators at runtime. A residual TNO layer has two costs. First, the DEC transports d^{k-1} , δ^{k+1} , and $\Delta_k^{\uparrow, \downarrow}$ are sparse matrix–dense matrix multiplies, costing $O(h(s_\bullet + \lambda_\bullet))$. Second, the learned maps are shared channel mixes applied cellwise, costing $O(n_\bullet h^2)$. Thus

$$T_{\text{TNO}}^{(1)} = O(h(s_\bullet + \lambda_\bullet) + n_\bullet h^2). \quad (22)$$

For bounded-degree cell complexes, $s_\bullet + \lambda_\bullet = O(n_\bullet) = O(n_0)$, so a *rigid* DEC TNO layer is $O(n_0 h^2)$.

For the MPNN in Eqs. (20) and (21), let n_0 be the number of vertices and m the number of directed edges; edge features are lifted to width w by the encoder, so they enter the layer at width w . The edge message W_m costs $O(mw^2)$, aggregation costs $O(mw)$, and the two-layer node update costs $O(n_0 w^2)$. Hence

$$T_{\text{MPNN}}^{(1)} = O((m + n_0)w^2), \quad (23)$$

which is $O(n_0 w^2)$ under bounded degree.

Both layers are quadratic in their channel width: $O(n_0 w^2)$ for MPNN and $O(n_0 h^2)$ for rigid DEC TNO, but the cost is accrued in different places. An MPNN spends $O(w^2)$ *per edge* in the message MLP; a rigid DEC TNO spends only $O(h)$ *per incidence* in transport and $O(h^2)$ *per cell* across all ranks in channel mixing. The extra factor $n_\bullet$ instead of n_0 is the cost of carrying edge, face, and higher-rank cochains as native state variables, which is what prescribes typed cross-rank routing. If instead the copresheaf realization is used with incidence-specific dense fiber maps $\rho_{y \rightarrow x} \in \mathbb{R}^{h \times h}$ (as in the CopresheafGIN variant of [37]; their GCN/Sage variants restrict $\rho_{y \rightarrow x}$ to be diagonal, retaining the $O(s_\bullet h)$ cost), transport raises to $O(s_\bullet h^2)$, matching the per-edge $O(w^2)$ scaling of MPNN message MLPs at common width.

E.3 Official Baseline Implementations: RIGNO, GAOT, HOGNN

For RIGNO [64], GAOT [86], and HOGNN [56], we use the authors' official implementations. These are adopted from published optimizer settings; we tune only the epoch budget and per-dataset width on the validation split; test numbers come from the best-validation checkpoint consistent with our models' evaluation protocol.

GAOT [86] is evaluated on both the RIGNO Tab. 1 and GAOT Tab. 2a benchmarks as it is the strongest published unstructured-mesh operator outside the methods evaluated by RIGNO Tab. 1, and

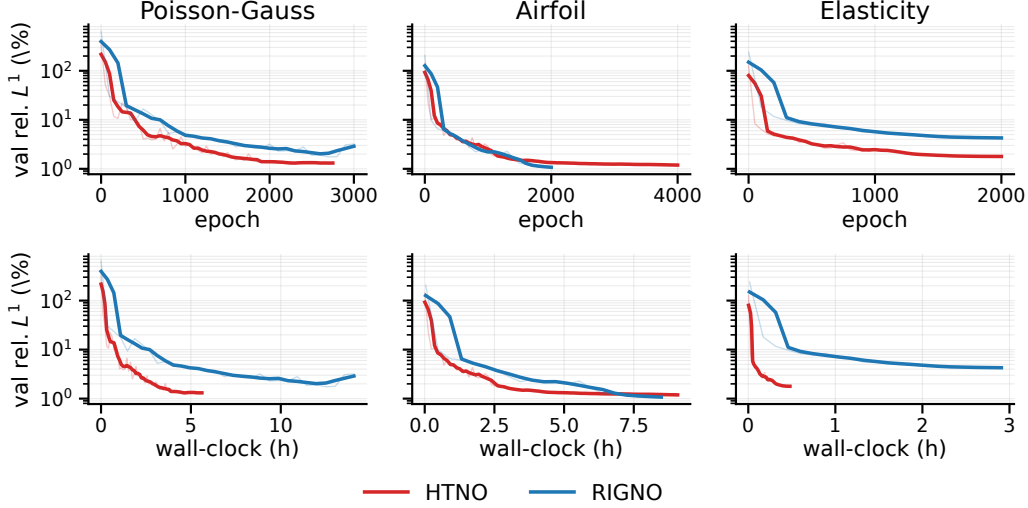


Figure 15: Validation relative L^1 vs. training epochs (left column) and cumulative wall-clock hours (right column) for HTNO and RIGNO on the three RIGNO steady-state benchmarks. Both models were trained under their respective official optimizer/batch settings; final test numbers correspond to the entries in Tab. 1.

it shares the same problem setting. We follow the upstream optimizer (AdamW with cosine-decay, peak 10^{-3} , weight decay 10^{-5} , gradient clipping 1.0) and its effective batch of 64 (data-parallel over four GH200). We extend the epoch budget to 2,000 with patience 300 (the upstream configurations use 100 and 1,000 for PG and Elasticity, respectively); due to best-validation checkpoint selection, this remains a safety margin to assure convergence.

We note that the public GAOT release does not match the paper’s Table 1 numbers on the GAOT suite itself (e.g. NACA0012 12.07% vs. paper 6.81%); this has been observed in issue #1 of the public repository that not all results are reproducible: for instance, they use 2048 train samples instead of 1024 (RIGNO / our protocol) on Poisson Gauss. GAOT entries are therefore a head-to-head comparison under identical experimental conditions, not as a reproduction of the paper’s reported results.

RIGNO [64] convergence observations. In contrast to the RIGNO multi-level physical/regional graph, HTNO arranges TNO blocks as a V -cycle over a sequence of coarsened cell complexes $K_0 \leftarrow \dots \leftarrow K_L$ connected by degree-preserving restriction and prolongation maps R_ℓ^k, Π_ℓ^k (Sec. 4): fine-level blocks capture short-range cross-degree physics, while coarse-level blocks, acting on the same cochain types after transfer, propagate long-range information. In addition to a superior performance across numerous benchmarks (see Sec. 5), we observe notably faster convergence. Fig. 15 depicts two qualitative observations from training: on these three benchmarks, HTNO reaches lower validation error in notably fewer epochs and in less wall-clock time than RIGNO, except Airfoil, where the two are comparable. Field-level prediction comparisons for these benchmarks are given in Sec. G.1.

HOGNN [56] is a closely related baseline, in a spirit that shares motivations with TNO. Both lift signals beyond the vertices using DEC operators, providing an informative head-to-head comparison. We run it on every 2D benchmark with a matching TNO or HTNO entry: PG, AF, Elasticity, NACA0012/2412, RAE2822, PCS, and the synthetic-topology family (Sec. 5.4). We use the upstream optimizer (AdamW with cosine-decay, peak 5×10^{-4} , weight decay 10^{-4} , gradient clipping 1.0), 300 epochs, patience 200, and the identical Delaunay graph construction as for TNO, HTNO, and RIGNO; per-dataset tuning is restricted to hidden width. EmmiWing is excluded because the public release is 2D only, and porting its DEC operator stack to 3D surfaces is outside our scope.

Results. Tab. 6 lists test relative L^1 error for HOGNN on every benchmark with a matching TNO or HTNO entry. HOGNN does not match the leading method in any of the cells evaluated; we therefore consider it not well-suited for these domains, as the authors primarily study applications to electrostatics.

Table 6: HOGNN reproduction across all 2D benchmarks: test relative L^1 (%); lower is better, **bold**: best per column. “Best of ours” is the smaller of TNO and HTNO from the cited table.

	Tab. 1			Tab. 2a			Tab. 3a			Tab. 3b	
Method	Poisson-Gauss	Airfoil	Elasticity	NACA0012	NACA2412	RAE2822	PCS	projected	native rank-2	Darcy	Adv-Diff.
Best of ours	1.03	1.11	1.70	3.77	3.92	3.92	0.52	5.42	4.88	4.87	5.20
HOGNN	33.78	15.64	21.24	9.36	10.33	9.28	50.44	14.84	15.97	12.04	13.23

F Experimental Details

We provide additional implementation details for our models, and baselines. TNO and HTNO are implemented in JAX [20]. We use the author’s official implementations (see Sec. E), which are often run in Pytorch [68]; we configure these with the same dataloaders our JAX models use to guarantee consistency. Implementations will be published upon acceptance. All models are run on a cluster of nodes consisting of 4 NVIDIA GH200s (96GB VRAM). The maximum compute budget on larger datasets (Emmiwing, PCS) is 4 GPUs simultaneously. Other steady-state benchmarks are run with 1 GPU per model.

F.1 Poisson-Gauss, Airfoil, Elasticity

Datasets. We use three datasets from the RIGNO benchmark [64]:

1. **Poisson-Gauss** (PG, Poisson with Gaussian sources on a random point cloud, 9,216 nodes, 1024/128/256 train/val/test, fixed positions);
2. **Airfoil Flow** (AF, compressible Euler flow with varying airfoil geometry at fixed far-field $M_\infty=0.8$, $\alpha=0^\circ$ [53, §4.4], 200×50 C-grid of quadrilateral elements (~ 11 K nodes) redistributed by [64] with 2048/128/256 train/val/test, variable positions);
3. **Elasticity** (hyper-elastic deformation of varying domains, ~ 1 K nodes, 1024/256/512, variable). We note that AF is, on closer inspection, an effectively single-condition transonic benchmark; we audit its regime spread against the GAOT airfoils in Sec. G.2.

Baselines. This section contains additional details regarding baselines and training protocols. We evaluate RIGNO [64], GAOT [86] and HOGNN [56] on the same splits using the authors’ official implementations (Sec. E.3).

Training protocol. For TNO, we report the best result from per-dataset tuning around a harmonic baseline (12 layers, hidden 128, $r=8$ harmonic modes), varying depth, width, learning rate, neighborhood size, and Delaunay graph construction. HTNO uses dynamic k -means clustering ($K=32$ –128 regions), 18 layers (6+6+6 fine/coarse/fine), hidden 128–256 depending on dataset scale. Training uses AdamW with cosine-decay (peak 5×10^{-4}), gradient clipping at 1.0, and early stopping with patience 200–500. Models train for 2,000–4,000 depending on convergence. The same protocol applies to the other benchmarks below.

F.2 Compressible Airfoils and Poisson-with-sines

Datasets. From the GAOT release [86] we use NACA0012/2412 and RAE2822 (~ 8 K-node airfoils, spanning subsonic, transonic, and supersonic regimes) and Poisson-C-Sines (PCS, fixed 16,431-node mesh).

Baselines. HTNO and TNO are trained as in the Sec. F.1 protocol; RIGNO and GAOT are evaluated on identical splits using the authors’ official implementations (Sec. E.3).

Splits. The GAOT public release contains per-dataset NetCDF files but does not publish the train/val/test indices, making it challenging to reproduce Table 1. We thus reconstruct splits based on the sample counts declared in the paper. We follow the upstream data pipeline’s default contiguous-block convention,

$$\text{train} = [0, N_{\text{tr}}), \quad \text{val} = [N_{\text{tr}}, N_{\text{tr}} + N_{\text{val}}), \quad \text{test} = [N_{\text{tot}} - N_{\text{te}}, N_{\text{tot}}),$$

i.e. the test split is the last N_{te} samples of the source file, with a gap between val and test on the larger airfoil files. Counts per dataset are listed in Tab. 7.

Table 7: Reconstructed GAOT-suite splits. N_{tot} is the sample count of the public release; the 5000/128/256 airfoil train/val/test split matches [86, §D.2.1, p.36], drawn from a pool of 5384 samples per airfoil; PCS uses the maximal contiguous train slice the file admits.

Dataset	N_{tot}	N_{tr}	N_{val}	N_{te}
NACA0012	43,490	5000	128	256
NACA2412	47,925	5000	128	256
RAE2822	48,375	5000	128	256
Poisson-C-Sines	5,000	4616	128	256

Bluff-Body. The Bluff-Body benchmark from the same release is omitted: the public collection contains only 7 of the 10 paper pretraining shapes (Square, Cone, and Rectangle-L are absent, and the paper’s 5-Ellipse pretraining set is replaced in the release by Ellipse-1/2 plus other shapes designated in the paper itself as fine-tuning geometries); we thus omit this evaluation due to incompleteness.

F.3 Large-Scale Surface PDEs: EmmiWing

Dataset and resolution protocol. EmmiWing [66] provides 29,727 steady-state compressible-RANS wing surfaces with raw meshes of 244K–426K nodes (variable across geometries). We use the official convex-hull-peeling split (ID-random, OOD-extrapolation, OOD-interpolation; 25,674/999/2,992 train/val/test, with 62 erroneous cases excluded). Following AB-UPT [2], we adopt a 65,536-geometry / 16,384 sampling protocol: each raw mesh is once FPS-resampled to 65K shared query points, and a fresh 16,384-node subset is drawn each training step. This follows established conventions in the literature: in [66], (PointNet, Transolver, ViT) also train at 16,384 surface points; AB-UPT [2] uses the 65,536→16,384 supernode/anchor pattern that we follow most closely.

Deviations from the published protocol. We evaluate on the 65K FPS subset, where [66, 2] evaluate on the full 244K–426K raw mesh via chunked inference. Aggregate- L^1 numbers are therefore not directly comparable to the published Table 2 of [66]; the comparisons in Tab. 2b are between our four models trained and evaluated under the same regime. Closing this gap would require equipping the topological backbone with a neural-field-style decoder that decouples prediction resolution from the fixed shared mesh, which we leave as future work.

Training protocol. In addition to HTNO and RIGNO-18, we report Transolver [87] (the slice-attention transformer designed for unstructured meshes), a pre-norm self-attention Transformer [83] with sinusoidal coordinate-based positional encoding, and a PointNet-style baseline [71] (per-point MLPs with global max-pool, implemented as pre-norm residual blocks). All models are trained under matched compute budgets with AdamW, peak learning rate 5×10^{-4} on a cosine-decay schedule, gradient clipping at 1.0, and early stopping on validation L^1 .

Per-channel results. Pressure is recovered to high fidelity by every model ($\leq 0.30\%$); the wall-shear-stress channels dominate the spread. HTNO leads on every channel; Transolver is the consistent runner-up, 0.2–0.3 pp behind on each τ component (see Tab. 2b).

F.4 Anisotropic Darcy with Per-Face Random Tensor Orientation

This section details the simulation protocol, dataset construction, and full result matrix for the rank-input study summarized in Sec. 5.3.

Synthetic-topology benchmark. The dataset family used in Sec. 5.4 is a 2D planar mesh distribution with a square outer boundary and zero, one, or two interior holes ($\sim 1,000$ nodes per mesh) under standard boundary jitter and adaptive mesh-density variation. The data layout is 100 unique meshes $\times$ 50 samples per mesh (each dataset has (4,000/500/500) train/val/test splits).

Anisotropic Darcy with face-valued κ -tensor. We solve

$$-\nabla \cdot (\kappa(x) \nabla u) = f, \quad u|_{\partial\Omega_{\text{outer}}} = 0, \quad u|_{\partial\Omega_{\text{hole},k}} = g_k \sim \mathcal{U}[0.5, 1.5],$$

with $\kappa(x)$ a DG0 (piecewise-constant per triangle) symmetric 2×2 tensor field reconstructed from a per-face orientation angle drawn iid per triangle, $\phi_t \sim \mathcal{U}[0, \pi)$:

$$\kappa_{\text{face}}[t] = R(\phi_t) \text{diag}(\kappa_{\parallel}, \kappa_{\perp}) R(\phi_t)^{\top}, \quad \kappa_{\parallel}=4, \kappa_{\perp}=1 \text{ (anisotropy ratio 4)}.$$

Concretely $\kappa_{xx}[t]=\kappa_{\perp} + (\kappa_{\parallel}-\kappa_{\perp}) \cos^2 \phi_t$, $\kappa_{yy}[t]=\kappa_{\perp} + (\kappa_{\parallel}-\kappa_{\perp}) \sin^2 \phi_t$, $\kappa_{xy}[t]=(\kappa_{\parallel}-\kappa_{\perp}) \cos \phi_t \sin \phi_t$. The forcing f is a superposition of 1–3 signed Gaussian bumps with amplitudes $a_i \sim \mathcal{U}[1.0, 2.5]$, $\sigma=0.25$, and the same mesh-adaptive minimum-width constraint used in Sec. 5.4. The variational form is assembled in FEniCSx [9] with P_1 Lagrange for u and DG0 for the three κ -tensor channels; the bilinear form expands the inner product directly to avoid UFL rank-mismatch on the asymmetric off-diagonal term.

Significance. The face channel $\cos(2\phi_t)$ has $\text{Var}[\cos(2\phi_t)]=\frac{1}{2}$ per face under $\phi_t \sim \mathcal{U}[0, \pi)$, but population mean $\mathbb{E}[\cos(2\mathcal{U}[0, \pi))]=0$. The vertex-mean projection (incident-face average over 5–6 neighbors) therefore concentrates further toward 0 at every vertex: the projected channel is near-constant noise. Empirically the face channel has mean $=-0.003$, std $=0.696$, matching the analytic prediction. **Vertex projection is therefore lossy by construction;** the operator’s coefficient field cannot be reconstructed from any vertex-supported representation.

Model input. Models receive $c=5$ vertex channels $[f, \mathcal{K}_{\partial}, g, \kappa_{\parallel}, \kappa_{\perp}]$ (the last two are global scalars carried to every vertex), an edge channel $c_e \in \mathbb{R}^{|E|}$ (face-incident mean of $\cos(2\phi_t)$), and a face channel $c_f \in \mathbb{R}^{|F|}$ ($\cos(2\phi_t)$ per triangle). For TNOs, c_e is routed through the rank-1 cochain channel and c_f through the rank-2 channel. The *projected* experiments instead replace the rank-aux channels with their vertex-mean projection appended as channels 6–7 of the vertex- c feature, with native rank-aux ingestion disabled.

Training protocol. All cells share one backbone: standard TNO with sheaf transport, harmonic basis, hidden 192, dropout 0.20, AdamW with cosine-decay (peak $5 \times 10^{-4} \rightarrow 10^{-5}$, weight decay 10^{-4}), gradient clipping 1.0, batch size 16, 300 epochs. The two MPNN baselines do not have sheaf nor harmonic components; for parameter-matched experiments, MPNN uses a width $w=272$ to reach $\sim 5.35\text{M}$ parameters (vs. $\sim 5.0\text{M}$ for TNO).

F.5 Ablations

We instantiate two scalar PDEs whose flux carries non-trivial curl on this distribution: Darcy ($-\nabla \cdot (\kappa \nabla u) + \sigma u = f$ with anisotropic κ), and conservative advection–diffusion, both assembled in FEniCSx [9] with the same variational forms used in the rank-input study. Within each mesh block, only the per-hole Dirichlet BC values vary across the 50 samples (drawn $\sim \mathcal{U}[0.5, 1.5]$); the source field, diffusivity tensor, and advection scale are held fixed, so topology is controlled for.

Backbone. All ablation variants share one backbone: hidden dim 192, MLP messages, $r_{\max}=1$, dropout 0.20, AdamW with cosine-decay (peak 5×10^{-4} , weight decay 10^{-4}), gradient clipping at 1.0, batch size 16, 300 epochs, rigid 2D augmentation enabled, single GH200. Variants differ only in the sheaf-transport and the harmonic-basis input. The two MPNN baselines use the same backbone with sheaf and harmonic disabled; the param-matched MPNN uses width $w=272$ to reach $\sim 5.35\text{M}$ parameters (vs. 5.0M for TNO).

G Extended Experimental Results

Next, we collect qualitative field-level comparisons for the steady-state benchmarks of Sec. 5 and a per-dataset regime audit that explains why the TNO/RIGNO ranking inverts between airfoil families.

G.1 Qualitative comparisons on the steady-state suites

RIGNO-suite (Airfoil, Elasticity). Fig. 16 shows field-level predictions on a median-test- L^1 sample of each dataset for TNO, RIGNO, and HTNO. On AF, all three architectures recover the ground-truth density field to within $\sim 1\text{--}3\%$ relative L^1 , with residuals concentrated near the airfoil tip and along the wake; on Elasticity, the residuals localize on the loaded notch boundary. The visual closeness of the three error maps on AF mirrors the table-level finding (Tab. 1) that TNO and RIGNO-18 are largely indistinguishable on this benchmark. This can be attributed to the dataset’s simple regime (see Sec. G.2).

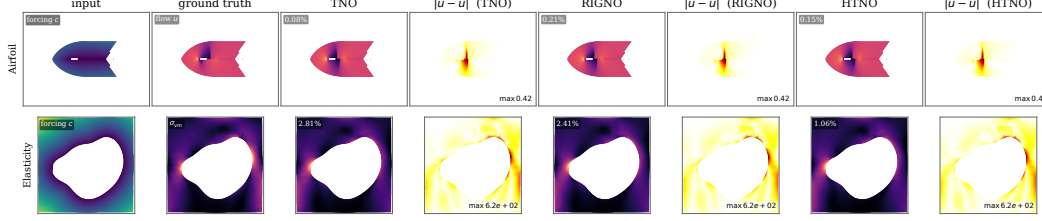


Figure 16: Qualitative comparison on the RIGNO-suite steady-state benchmarks (Airfoil, Elasticity). Each row shows one median-test- L^1 sample for the dataset; columns: input forcing, ground-truth field, and per-architecture prediction $\hat{u}$ followed by absolute error $|\hat{u} - u|$ for TNO, RIGNO, and HTNO. The three error panels in a row share a common color scale, with the shared max stamped on each panel so residual magnitudes are directly comparable across architectures.

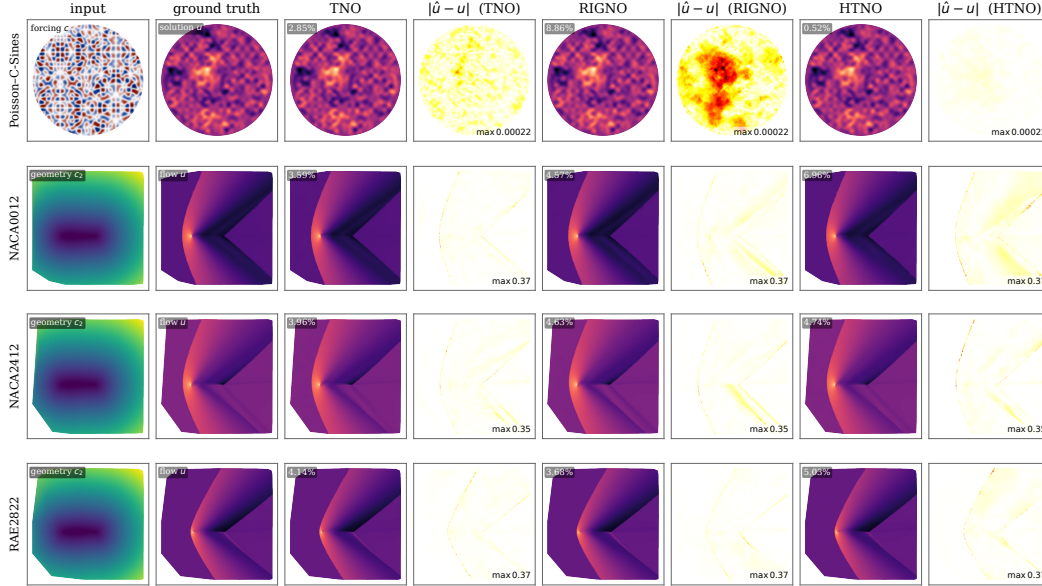


Figure 17: Qualitative comparison on the GAOT-suite benchmarks (Poisson-C-Sines, NACA0012, NACA2412, RAE2822). Each row shows one median-test- L^1 sample for the dataset; columns: model input, ground-truth field, and per-architecture prediction $\hat{u}$ followed by absolute error $|\hat{u} - u|$ for TNO, RIGNO, and HTNO. Per-row error panels share a common color scale, allowing residual magnitudes to be directly comparable across architectures. Triangulation on the airfoil rows uses a vertex-local long-edge filter to drop spurious convex-hull bridges across the wing void.

GAOT-suite (PCS, NACA0012/2412, RAE2822). Fig. 17 shows the analogous comparison on the four single-file steady-state datasets of [86]. PCS (top row) is dominated by a high-frequency multiscale sinusoidal forcing pattern, which HTNO and GAOT can represent well. The three airfoil rows expose the regime-sweep difficulty of this benchmark: the operators concentrate residual along the boundary layer and—for the supersonic RAE2822 sample—along the captured shock, where flatter message-passing baselines visibly under-resolve the discontinuity while TNO’s harmonic + DEC structure appears to track it more tightly. Per-row error panels share a common color scale, so RIGNO and HTNO residuals are directly comparable.

G.2 Differences in airfoils across datasets

TNO marginally underperforms RIGNO-18 on AF and outperforms it by 19–32% relative test- L^1 on the GAOT. This is due to how the benchmarks differ in what their generators sweep across samples (Tab. 8): [53] fixes the AF far-field at $M_\infty=0.8$, $\alpha=0^\circ$ and varies only the airfoil geometry, exposing a single distance-to-boundary field as conditioning (RIGNO [64] adopts this dataset directly); [86] draws each NACA0012/2412/RAE2822 sample at a different (M, α) with $M \in [0.5, 1.4]$ and $\alpha \in [0.5^\circ, 5.0^\circ]$ on per-sample meshes, exposing M and α as conditioning scalars.

Table 8: What varies across samples for the two airfoil benchmark families (published quantities).

Dataset family	M_∞	α	conditioning channels	varies per sample
AF [53]	0.8 (fixed)	0° (fixed)	distance field	airfoil geometry only
GAOT airfoils [86]	[0.5, 1.4]	[0.5° , 5.0°]	M , α (scalars)	(M , α , mesh)

H Related Work

Neural operators. Operator learning (OL) approximates maps between infinite-dimensional function spaces, with the aim of producing models that generalize across initial conditions, coefficients, forcings, and discretizations [48, 19, 47, 8]. *DeepONet* [60] and its physics-informed extensions [34, 46] learn a branch–trunk factorization of the operator with a universal approximation guarantee in function spaces. The *neural operator* family of Li et al. [51] instead realizes the operator through learned integral kernels: the Graph Neural Operator (GNO) approximates these kernels by message passing on sampled point clouds, while the Fourier Neural Operator (FNO) [52] replaces the kernel by a parameterized spectral multiplier on a regular grid, with subsequent factorized [79], wavelet [81], and convolutional [73] variants. Physics-informed neural networks complement these by deriving losses from the governing PDEs [55, 85, 90], and a separate line studies OL via model reduction [12].

Neural operators on irregular geometries. PDEs on irregular domains have motivated a body of operator-learning extensions beyond regular Cartesian grids. Geo-FNO [53] learns a deformation from the physical mesh to a uniform latent grid on which the FNO is applied; DAFNO [59] embeds geometry through smoothed indicator masks; and Beyond-Regular-Grids [57] evaluates truncated spectral transforms directly on arbitrary point sets. GINO [54] couples a graph encoder/decoder with an FNO on a uniform latent grid, scaling operator learning to large 3D meshes. Mesh-based simulators include MeshGraphNet [69], which propagates information on the simulation mesh together with a separate world-edge graph, and RIGNO [64], which uses message passing on multiscale graphs; GAOT [86] pairs a multiscale attentional graph encoder with a transformer processor on a latent grid. Transformer-based operators [50, 39, 87, 23, 25] treat unstructured nodes as tokens and attend either across all points or across learned cluster representatives, with universal physics transformers [1, 2] and PDE foundation models [40] extending this idea to large-scale 3D aerodynamics [66].

A complementary line treats geometry intrinsically: DIMON [88] learns the operator across a diffeomorphic family of domains, geometric NOs on Riemannian manifolds [72, 26] and gauge-equivariant intrinsic operators [28, 27] build manifold structure into the architecture, and Neural Green’s Functions [89] parameterize geometry-dependent fundamental solutions. What separates these methods from TNO is that physical quantities are still represented as functions on *points* (graph nodes, deformed grid samples, or tokens); cross-dimensional differential structure is recovered indirectly through learned kernels. TNOs explicitly address this gap: by keeping cochains of all degrees as first-class objects and routing information through fixed boundary, coboundary, and Hodge operators, geometric type and the conservation identities $d \circ d = 0$ become structural rather than learned.

Discrete exterior calculus and structure-preserving discretizations. The mathematical foundation behind TNOs is the discretization of differential forms. Discrete exterior calculus [44, 32, 31, 30, 70, 11] models physical quantities as cochains on oriented cells and replaces gradient, curl, and divergence by signed boundary operators that satisfy a discrete Stokes’ theorem. The same viewpoint underlies Whitney forms [17, 18], mixed and Nédélec finite elements [65, 42], mimetic finite differences [45, 58], and the finite element exterior calculus of Arnold et al. [5, 6], Arnold [3], all of which preserve the de Rham complex and the algebraic identity $B_k B_{k+1} = 0$ at the discrete level. TNOs inherit this structure: the cellular operators $d^k = B_{k+1}^\top \delta^k$, and the Hodge Laplacian Δ_k used in our layers are exactly the DEC operators on the underlying complex.

Topological deep learning (TDL) & architectures for PDE learning. TDL [67, 36] extends graph neural networks by allowing message passing across cells of multiple ranks (vertices, edges, faces, and higher). Simplicial complex networks generalize the Weisfeiler–Leman test to higher-order incidences [15], CW networks operate on regular cell complexes [14]. Combinatorial complexes [35] unify these under a single neighborhood scheme. Sheaf neural networks [38, 16] and copresheaf topological networks [37] attach learnable fiber maps to incidence relations, while Hodge Laplacian random walks [76, 84] and topology-aware GNNs [49], learning the hodge-star [77], provide founda-

tions. Despite the appeal of the cellular viewpoint for physics, topological architectures remain almost absent from the OL literature, and the closest prior works are graph-based and message-passing in flavor: SNN-PDE [29] uses simplicial convolutions for **time-dependent** PDEs. Trask et al. [80] learn Whitney-form-style metric data on a fixed graph to obtain a data-driven exterior calculus that enforces conservation. Finally, DEC-HOGNN [56] builds a higher-order GNN on DEC and FEEC primitives for boundary-value electromagnetic problems. While these methods exploit cell-level features, **they remain GNN-style architectures**. None define true function-space operators, expose the Hodge decomposition as an architectural component or address multi-degree mPDE systems where unknowns at different ranks couple simultaneously through d and δ . TNOs address these by fixing information flow to the DEC operators and by exposing the Hodge decomposition in the layer (Eq. (10)), so that exact, coexact, and harmonic channels carry physically meaningful components.